

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
Khoa Công nghệ Thông tin

BÁO CÁO ĐỒ ÁN 3
CÁC CHỦ ĐỀ MỚI TRONG NGHIÊN CỨU HỌC MÁY

Generative AI Meets Reinforcement Learning

ICML 2025 Tutorial – Benjamin Eysenbach & Amy Zhang

Môn học: Nhập môn Học Máy

Mã môn: CSC14005

Học kỳ: Học kỳ 2 – 2025/2026

Giảng viên: ThS. Lê Nhật Nam

MSSV	Họ và tên	Phần đóng góp
23120059	Trần Đình Luân	Tổng quan, kiến thức nền
23120063	Nguyễn Thành Nguyên	Nội dung chính – Phần 1
23120062	Trần Kim Ngọc	Nội dung chính – Phần 2
23120084	Nguyễn Mạnh Thắng	Demo thực nghiệm
23120064	Nguyễn Thiện Nhân	Đánh giá, kết luận

Mục lục

1	Tổng quan chủ đề	1
1.1	Giới thiệu bài toán	1
1.2	Động lực nghiên cứu	1
1.2.1	Giới hạn của Imitation Learning	1
1.2.2	Giới hạn của Reward Engineering	2
1.2.3	Sự hội tụ của hai lĩnh vực	2
1.3	Tầm quan trọng và tính thời sự	2
1.3.1	Ứng dụng thực tiễn	2
1.3.2	Generalization – bài toán còn bỏ ngỏ	3
1.4	Cấu trúc báo cáo	3
2	Kiến thức nền	4
2.1	Reinforcement Learning	4
2.1.1	Vòng lặp Agent–Environment: Bức tranh tổng thể	4
2.1.2	Định nghĩa bài toán RL: Markov Decision Process	5
2.1.3	Policy và Mục tiêu RL	5
2.1.4	So sánh RL và Supervised Learning	6
2.1.5	Value Functions và Bellman Equations	7
2.1.6	Policy Gradient	7
2.2	Generative Models	8
2.2.1	Tổng quan và Phân loại	8
2.2.2	Variational Autoencoder (VAE)	8
2.2.3	Generative Adversarial Network (GAN)	9
2.2.4	Diffusion Models	9
2.2.5	Flow Matching Models	10
2.2.6	Transformer và Autoregressive Models	11
2.3	Successor Measure	11
2.3.1	Ý nghĩa trực quan và ví dụ minh họa	12

2.3.2	Liên hệ với Q-function	12
2.3.3	Tính toán qua Bellman	13
2.3.4	Vị trí trong Tutorial	13
2.4	Bảng đối chiếu RL và Generative Modeling	13
3	Nội dung chính của tutorial	15
3.1	Generative Models in RL Engines	15
3.1.1	SL Objective vs RL Objective	15
3.1.2	Generative Models as World Models	16
3.1.3	Reward-Weighted Learning and the Score Function	18
3.1.4	Generative Models as Policies	20
3.1.5	Case Study: Diffusion-DICE	21
3.1.6	DDPO: Viewing Denoising as an MDP	23
3.2	Interaction as a Generative Model	24
3.2.1	Interaction as a Generative Process	25
3.2.2	Goal-Reaching and Occupancy Measures	26
3.2.3	Goal-Conditioned Behavioral Cloning	27
3.2.4	Dual RL and DICE	28
3.2.5	f-Policy Gradients and State-MaxEnt RL	30
3.2.6	Offline Goal-Conditioned RL and Technical Synthesis	31
3.3	Học Likelihoods	33
3.3.1	Successor Measure	33
3.3.2	Ước lượng Density Trực tiếp	33
3.3.3	Temporal Contrastive Learning	34
3.3.4	Fully-general RL từ Density Ratios	36
3.3.5	Proto-Successor Measure (PSM)	38
3.3.6	RLZero: Zero-shot RL từ Ngôn ngữ Tự nhiên	38
3.4	Data Tự Sinh	41
3.4.1	Bài toán Exploration	42
3.4.2	Empowerment	42

3.4.3	Skill Learning	43
	Phân tích Hạn chế và Đề xuất Cải tiến	46
4	Thực nghiệm và Kết quả	48
4.1	Thiết lập bài toán	48
4.2	Các phương pháp so sánh	48
4.3	Kết quả thực nghiệm	50
4.4	Mở rộng 1: Thực nghiệm trên Dataset 4 cụm (4-Cluster)	51
4.5	Mở rộng 2: Khảo sát siêu tham số Guidance Scale	51
4.6	Mô tả Demo	52
5	Đánh giá và Kết luận	54
5.1	Đánh giá tổng thể tutorial	54
5.1.1	Đóng góp khoa học chính	54
5.1.2	Tính nhất quán và mạch lạc của framework	54
5.2	Ưu điểm của hướng tiếp cận	55
5.2.1	Loại bỏ sự phụ thuộc vào reward engineering	55
5.2.2	Mở rộng năng lực biểu diễn của policy	55
5.2.3	Khả năng học không cần giám sát và dữ liệu tự thu thập	55
5.3	Hạn chế và thách thức còn tồn tại	56
5.3.1	Hạn chế về khả năng mở rộng (scalability)	56
5.3.2	Vấn đề phân phối lệch (distribution shift)	56
5.3.3	Phụ thuộc vào chất lượng reward model	56
5.3.4	Thách thức trong khái quát hóa (generalization)	57
5.4	Hướng nghiên cứu tương lai	57
5.4.1	Tích hợp nền tảng mô hình (foundation models) với RL	57
5.4.2	Cải thiện hiệu quả mẫu trong môi trường thực	57
5.4.3	Reward-free RL ở quy mô lớn	57
5.4.4	Lý thuyết về generalization trong RL	58
5.5	Nhận xét về ý nghĩa thực tiễn	58

5.6	Kết luận	58
-----	--------------------	----

1. Tổng quan chủ đề

1.1. Giới thiệu bài toán

Năm 1770, Wolfgang von Kempelen giới thiệu với châu Âu một cỗ máy biết chơi cờ mang tên “Mechanical Turk” – một thiết bị tinh vi với hàng chục bánh răng và đòn bẩy. Nó đánh bại Napoleon, Benjamin Franklin và vô số đối thủ khác. Hóa ra, toàn bộ bộ máy chỉ là vỏ bọc, bên trong có một kỳ thủ thật ngồi điều khiển mọi nước đi qua hệ thống gương phản chiếu.

Câu chuyện này phản ánh chính xác trạng thái của AI ngày nay. Phần lớn các hệ thống AI hiện đại – từ GPT-4, Gemini cho đến Stable Diffusion – đều được xây dựng trên nguyên tắc **maximum likelihood imitation**: mô hình học cách tái tạo hành vi của con người bằng cách tối đa hóa xác suất sinh ra dữ liệu giống dữ liệu huấn luyện. Phương pháp này rất mạnh, nhưng chứa đựng một giới hạn nền tảng không thể tránh khỏi:

$$\text{Khả năng của mô hình} \leq \text{Khả năng của dữ liệu huấn luyện} \quad (1)$$

Nếu tất cả dữ liệu đều sai, mô hình sẽ sai theo. Nếu chưa có con người nào giải được một bài toán, mô hình không thể học cách giải nó từ dữ liệu đó. Đây là bài toán cốt lõi mà tutorial *Generative AI Meets Reinforcement Learning* [7] đặt ra: làm thế nào để xây dựng AI có thể **vượt qua giới hạn của dữ liệu huấn luyện**?

Tutorial được trình bày tại ICML 2025 bởi Benjamin Eysenbach (Princeton University) và Amy Zhang (UT Austin), xoay quanh hai câu hỏi liên quan chặt chẽ:

- Reinforcement Learning và Generative AI – hai lĩnh vực tưởng như tách biệt – liệu có chia sẻ cùng cấu trúc toán học không?
- Nếu có, chúng ta có thể khai thác điều đó để xây dựng các hệ thống AI mạnh mẽ hơn như thế nào?

1.2. Động lực nghiên cứu

1.2.1. Giới hạn của Imitation Learning

Như đã trình bày, phần lớn AI hiện đại được huấn luyện bằng cách tối đa hóa khả năng tái tạo hành vi của con người. Cách tiếp cận này hiệu quả khi có đủ dữ liệu chất lượng cao và nhiệm vụ nằm trong phân phối dữ liệu. Tuy nhiên, nó gặp khó khăn

cơ bản khi bài toán đòi hỏi sự sáng tạo vượt ngoài dữ liệu, hoặc khi chưa có ai giải được bài toán đó trước đây.

Một hệ quả đáng lo ngại hơn nữa là hiện tượng *model collapse*: khi AI được huấn luyện đệ quy trên dữ liệu do chính AI sinh ra, chất lượng suy giảm theo cấp số nhân [16].

1.2.2. Giới hạn của Reward Engineering

Nền tảng của Reinforcement Learning là tín hiệu phần thưởng (reward). Tuy nhiên, hầu hết bài toán thực tế rất khó thiết kế reward function phù hợp – đây là mâu thuẫn cốt lõi: RL hứa hẹn tìm ra giải pháp tốt hơn con người, nhưng chính reward function lại đòi hỏi con người phải hiểu rõ vấn đề mới có thể thiết kế được.

Với bài toán dọn phòng robot, người thiết kế phải tự hỏi: nhặt một chiếc áo được bao nhiêu điểm? Xếp sách ngay ngắn được bao nhiêu? Nếu thiết kế sai, robot có thể học cách “gian lận” thay vì thực sự dọn dẹp – một hiện tượng được gọi là *reward hacking*.

1.2.3. Sự hội tụ của hai lĩnh vực

Trong nhiều thập kỷ, RL và Generative AI phát triển song song nhưng tương đối độc lập. Gần đây, giới nghiên cứu nhận ra rằng hai lĩnh vực này đang giải quyết cùng một bài toán từ hai góc nhìn khác nhau, và khai thác điểm chung này mở ra ba hướng giao thoa chính:

Bảng 1: Ba hướng giao thoa giữa Generative AI và RL.

Hướng	Ý nghĩa
GenAI $\rightarrow$ RL	Dùng generative models làm thành phần trong RL
RL $\rightarrow$ GenAI	Nhìn tương tác RL như một generative process
RL để train GenAI	Dùng RL để cải thiện cách huấn luyện generative models

1.3. Tầm quan trọng và tính thời sự

1.3.1. Ứng dụng thực tiễn

Sự kết hợp giữa Generative AI và RL có nhiều ứng dụng thực tiễn quan trọng [4] [13]:

Robotics: Diffusion policies đã đạt kết quả tốt nhất trong nhiều tác vụ điều khiển

robot. Kết hợp với RL, robot có thể vượt qua giới hạn của dữ liệu demonstration, tự học cách thực hiện các hành vi chưa từng được quan sát.

Behavioral Foundation Models: Tương tự như GPT là foundation model cho ngôn ngữ, các Behavioral Foundation Model – được học thông qua RL không có reward – có thể đóng vai trò foundation model cho hành động, một model dùng được cho nhiều tác vụ robot khác nhau mà không cần huấn luyện lại.

RLHF và AI alignment: Reinforcement Learning from Human Feedback (RLHF) – kỹ thuật cốt lõi làm cho ChatGPT “nghe lời” – là ứng dụng trực tiếp của RL để fine-tune generative models. Hiểu sâu mối liên hệ này giúp thiết kế các phương pháp alignment tốt hơn.

Khoa học và khám phá: AlphaFold dự đoán cấu trúc protein, AlphaProof giải bài thi toán Olympic – các hệ thống này đều dùng RL để tìm giải pháp mà không có dữ liệu tương ứng từ trước.

1.3.2. Generalization – bài toán còn bỏ ngỏ

Tutorial đặt ra một câu hỏi quan trọng về tương lai: điều gì làm cho generative image models và language models trở nên ấn tượng không phải là chúng ghi nhớ dữ liệu đã thấy, mà là chúng *generalize* – tạo ra output đúng trên những input chưa bao giờ thấy trong training.

Trong khi đó, RL agents ngày nay phần lớn vẫn chỉ ghi nhớ thay vì generalize. Tutorial đặt câu hỏi: tương tự như một generative image model có thể tạo ra ảnh sống động sau khi chỉ thấy một phần nhỏ các ảnh có thể, liệu ta có thể xây dựng RL agents có thể giải quyết các bài toán mới sau khi chỉ practice trên một phần nhỏ không gian bài toán đó? Đây chính là *generalization frontier* mà kết hợp Generative AI và RL hướng tới.

1.4. Cấu trúc báo cáo

Báo cáo được tổ chức thành các phần chính như sau:

- **Phần 2** trình bày kiến thức nền về Reinforcement Learning và Generative Models, các khái niệm cơ sở cần thiết để hiểu nội dung tutorial.
- **Phần 3** trình bày nội dung chính của tutorial, bao gồm Phần I (Generative Models trong RL), Phần II (Tương tác như một Generative Model), Phần III (Học Likelihoods) và Phần IV (Data Tự Sinh).
- **Phần 4** mô tả thực nghiệm và kết quả demo nhóm thực hiện.

- **Phần 5** đánh giá ưu nhược điểm của các phương pháp và thảo luận hướng nghiên cứu tương lai.

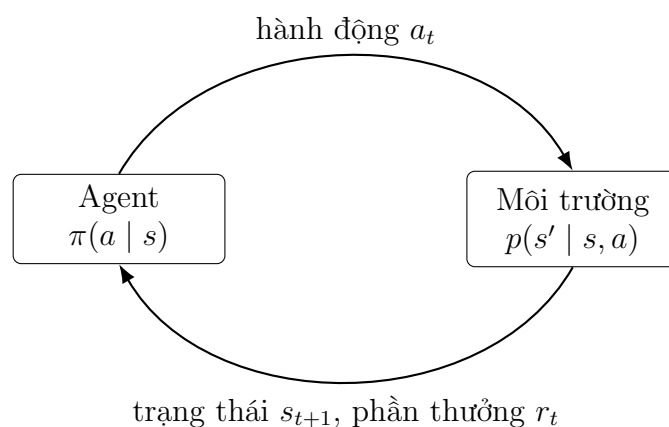
2. Kiến thức nền

2.1. Reinforcement Learning

2.1.1. Vòng lặp Agent–Environment: Bức tranh tổng thể

Trước khi đi vào công thức toán học, hãy hình dung RL hoạt động như thế nào theo trực giác. Trong RL, một **agent** (ví dụ: robot, chương trình chơi game, hệ thống điều khiển) liên tục tương tác với một **môi trường** (*environment*) theo vòng lặp lặp đi lặp lại:

1. Agent quan sát **trạng thái hiện tại** s_t của môi trường (ví dụ: hình ảnh màn hình game, vị trí các khớp robot).
2. Agent chọn một **hành động** a_t dựa trên *policy* (chiến lược) của mình (ví dụ: nhảy, đi thẳng, xoay trái).
3. Môi trường chuyển sang **trạng thái mới** s_{t+1} theo xác suất $p(s_{t+1} | s_t, a_t)$, đồng thời phát ra **tín hiệu phần thưởng** $r_t = r(s_t, a_t)$ (ví dụ: +1 mỗi khi xóa được một dòng trong Tetris, -1 khi robot ngã).
4. Lặp lại từ bước 1.



Hình 1: Vòng lặp agent–environment trong RL. Agent gửi hành động cho môi trường; môi trường phản hồi trạng thái mới và phần thưởng. Mục tiêu agent là học policy π tối đa hóa tổng phần thưởng tích lũy.

Điểm khác biệt căn bản so với Supervised Learning là: **không có “đáp án đúng” nào được cung cấp sẵn**. Agent phải tự khám phá thông qua thử-và-sai, và tín hiệu học duy nhất là phần thưởng thua thốt từ môi trường. Điều này đồng thời là điểm mạnh (agent có thể tìm ra chiến lược vượt trội hơn bất kỳ dữ liệu huấn luyện nào) và là thách thức chính của RL.

2.1.2. Định nghĩa bài toán RL: Markov Decision Process

Bài toán RL được hình thức hóa bằng khung **Markov Decision Process (MDP)**, bao gồm năm thành phần:

$$\text{MDP} = (\mathcal{S}, \mathcal{A}, p, r, \gamma) \quad (2)$$

Bảng 2: Các thành phần của MDP, minh họa qua game Tetris.

Ký hiệu	Tên	Ví dụ (Tetris)
$\mathcal{S}$	Không gian trạng thái	Mọi cấu hình bảng có thể
$\mathcal{A}$	Không gian hành động	{trái, phải, xoay, thả}
$p(s' s, a)$	Dynamics (hàm chuyển trạng thái)	Xác suất khối rơi vào ô nào
$r(s, a)$	Hàm phần thưởng	+1 mỗi dòng xóa được
$\gamma \in [0, 1)$	Hệ số chiết khấu	Thường từ 0.9 đến 0.999

Giả định **Markov** là nền tảng quan trọng của khung này: trạng thái hiện tại s_t chứa đủ thông tin để quyết định hành động tiếp theo, không cần nhớ toàn bộ lịch sử. Nói cách khác, “quá khứ” chỉ ảnh hưởng đến tương lai thông qua trạng thái hiện tại. Nhờ tính Markov, ta có thể dùng các phương trình đệ quy (Bellman equations) để tính toán hiệu quả thay vì phải xét toàn bộ lịch sử.

2.1.3. Policy và Mục tiêu RL

Policy π là chiến lược ra quyết định của agent – một hàm ánh xạ từ trạng thái sang hành động (hoặc phân phối trên hành động):

$$\pi(a | s) = \text{xác suất chọn action } a \text{ khi ở state } s$$

Ví dụ: policy của người chơi Tetris giỏi sẽ gán xác suất cao cho hành động “xoay” khi nhìn thấy khối L ở vị trí nhất định.

Một **trajectory** (quỹ đạo) $\tau = (s_0, a_0, s_1, a_1, \dots)$ là chuỗi trạng thái và hành động theo thời gian – về bản chất là một “ván chơi” hoàn chỉnh. **Discounted return**

$R(\tau)$ là tổng phần thưởng có chiết khấu theo thời gian:

$$R(\tau) = \sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \quad (3)$$

Hệ số chiết khấu $\gamma < 1$ giải quyết hai vấn đề: nó đảm bảo tổng phần thưởng hội tụ (không vô hạn), đồng thời phản ánh nguyên tắc kinh tế học rằng phần thưởng gần hơn có giá trị hơn (giống như tiền ngày hôm nay có giá trị hơn tiền ngày mai).

Mục tiêu RL là tìm policy π tối đa hóa expected return – tức phần thưởng tích lũy kỳ vọng trên tất cả các trajectory có thể xảy ra:

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} [R(\tau)] = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \right] \quad (4)$$

Ký hiệu $\tau \sim \pi$ nghĩa là trajectory được sinh ra bởi chính policy π tương tác với môi trường: tại mỗi bước t , agent chọn $a_t \sim \pi(\cdot \mid s_t)$, môi trường phản hồi $s_{t+1} \sim p(\cdot \mid s_t, a_t)$.

2.1.4. So sánh RL và Supervised Learning

Sự khác biệt căn bản giữa RL và Supervised Learning (SL) nằm ở nguồn gốc dữ liệu:

$$\text{SL: } \max_{\theta} \mathbb{E}_{(s,a) \sim \mathcal{D}} [\log \pi_{\theta}(a \mid s)] \quad (5)$$

$$\text{RL: } \max_{\theta} \mathbb{E}_{\tau \sim \pi_{\theta}} [R(\tau)] \quad (6)$$

Trong SL, dữ liệu đến từ dataset cố định $\mathcal{D}$ và không phụ thuộc vào tham số đang học. Trong RL, trajectory được sinh ra bởi chính policy π_{θ} đang được tối ưu – khi θ thay đổi, cả phân phối hành động lẫn phân phối trạng thái được ghé thăm đều thay đổi theo. Đây chính là điểm mạnh (agent có thể khám phá) nhưng cũng là thách thức chính (dữ liệu *non-stationary*) của RL.

Ví dụ trực quan: nếu dùng SL để học chơi Tetris trên dataset từ người chơi mới bắt đầu, model sẽ không thể giỏi hơn người đó. Nhưng với RL, agent tự tạo dữ liệu thông qua tương tác và có thể tìm ra chiến lược tốt hơn bất kỳ dữ liệu nào trong dataset [7].

2.1.5. Value Functions và Bellman Equations

Để tối ưu $J(\pi)$, cần biết “từ trạng thái này, hành động nào dẫn đến return cao nhất?” – đây là vai trò của **value functions**.

Q-function $Q^\pi(s, a)$ cho biết return kỳ vọng khi bắt đầu từ cặp (s, a) và sau đó theo policy π :

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \mid s_0 = s, a_0 = a \right] \quad (7)$$

Diễn giải đơn giản: $Q^\pi(s, a)$ trả lời câu hỏi “Nếu tôi đang ở trạng thái s , thực hiện hành động a , rồi từ đó trở đi luôn theo policy π , tôi kỳ vọng nhận được bao nhiêu phần thưởng tổng cộng?”

Q-function thỏa **phương trình Bellman** – quan hệ đệ quy cơ bản của RL. Phương trình này phát biểu rằng giá trị hiện tại bằng phần thưởng ngay lập tức cộng với giá trị tương lai được chiết khấu:

$$Q^\pi(s, a) = \underbrace{r(s, a)}_{\text{thưởng ngay}} + \gamma \mathbb{E}_{s' \sim p(\cdot | s, a), a' \sim \pi(\cdot | s')} \left[\underbrace{Q^\pi(s', a')}_{\text{giá trị tương lai}} \right] \quad (8)$$

Phương trình Bellman quan trọng vì nó biến bài toán tối ưu hóa vô hạn chiều thành bài toán có thể giải lặp đi lặp lại (dynamic programming). Q-function xuất hiện xuyên suốt tutorial – trong diffusion policies (dùng Q để chọn hành động), trong temporal contrastive learning ($Q \approx \log \text{likelihood ratio}$), và trong zero-shot RL (tính Q từ biểu diễn học được).

2.1.6. Policy Gradient

Để tối đa hóa $J(\pi_\theta)$, ta lấy gradient theo tham số θ và thực hiện gradient ascent. Định lý Policy Gradient cho kết quả:

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[R(\tau) \cdot \sum_t \nabla_\theta \log \pi_\theta(a_t | s_t) \right] \quad (9)$$

Trực giác của policy gradient rất tự nhiên: số hạng $\nabla_\theta \log \pi_\theta(a | s)$ đẩy tham số theo hướng tăng xác suất của hành động a tại trạng thái s ; nhân với $R(\tau)$ nghĩa là ta tăng xác suất của những hành động đã dẫn đến return cao, và giảm xác suất của những hành động dẫn đến return thấp.

Số hạng $\nabla_{\theta} \log \pi_{\theta}(a | s)$ được gọi là **score function** của policy. Đây là cầu nối kỹ thuật quan trọng nhất giữa RL và generative modeling: trong score-based diffusion models, đại lượng tương tự $\nabla_x \log p(x)$ được dùng để hướng dẫn quá trình sinh mẫu. Hai bài toán – tối ưu policy và sinh mẫu từ generative model – đều có thể được giải qua cùng một công cụ toán học: **theo dõi gradient của log-probability**.

2.2. Generative Models

2.2.1. Tổng quan và Phân loại

Generative model là mô hình có khả năng (1) tạo ra mẫu mới từ một phân phối học được, và (2) ước lượng likelihood (xác suất) của các mẫu đã cho. Khác với discriminative models (phân loại đầu vào), generative models học phân phối xác suất $p(x)$ của dữ liệu và có thể sinh ra mẫu mới $x \sim p(x)$.

Ý tưởng chung của các generative models mạnh nhất hiện nay là thực hiện **tính toán lặp đi lặp lại** để tạo ra mẫu:

- **VAE** và normalizing flow biến đổi vector nhiễu qua nhiều lớp mạng neural.
- **Diffusion model** lặp đi lặp lại quá trình khử nhiễu qua hàng trăm bước.
- **Transformer** sinh token từng bước một theo thứ tự.
- **Flow matching model** tích phân một vector field để biến đổi phân phối.

Đặc điểm “nhiều bước” này không phải ngẫu nhiên – nó phản ánh một nguyên lý sâu xa: các phân phối phức tạp rất khó nắm bắt trong một bước duy nhất, nhưng lại có thể được xây dựng dần dần qua nhiều bước đơn giản hơn. Tutorial chỉ ra rằng **đây chính là điểm tương đồng với RL**: một RL agent cũng tạo ra kết quả thông qua nhiều bước tương tác với môi trường.

2.2.2. Variational Autoencoder (VAE)

VAE là một trong những generative model đầu tiên học được không gian tiềm ẩn (latent space) ý nghĩa. VAE gồm hai thành phần: encoder $q_{\phi}(z | x)$ nén dữ liệu x vào vector tiềm ẩn z , và decoder $p_{\theta}(x | z)$ tái tạo dữ liệu từ z . Quá trình sinh mẫu mới đơn giản là lấy mẫu $z \sim \mathcal{N}(0, I)$ rồi cho qua decoder.

VAE được huấn luyện bằng cách tối đa hóa ELBO (Evidence Lower BOund):

$$\mathcal{L}_{\text{VAE}} = \mathbb{E}_{q_{\phi}(z|x)}[\log p_{\theta}(x | z)] - D_{\text{KL}}(q_{\phi}(z | x) \parallel \mathcal{N}(0, I)) \quad (10)$$

Số hạng đầu khuyến khích decoder tái tạo tốt; số hạng KL regularize không gian tiềm ẩn về phân phối Gaussian chuẩn.

Kết nối với RL: Trong skill learning (Phần IV của tutorial), biến tiềm ẩn z của VAE tương đồng chính xác với “skill label” z – một biến điều kiện quyết định loại hành vi agent thực hiện. Cả hai đều nén thông tin về một tập hành vi phức tạp vào một không gian nhỏ hơn có thể tìm kiếm được.

2.2.3. Generative Adversarial Network (GAN)

GAN gồm hai mạng thi đấu với nhau theo dạng trò chơi minimax: *generator* G cố gắng sinh mẫu giống thật, *discriminator* D cố gắng phân biệt mẫu thật với mẫu giả:

$$\min_G \max_D \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim \mathcal{N}(0, I)} [\log(1 - D(G(z)))] \quad (11)$$

Khi đạt cân bằng Nash, generator học được phân phối dữ liệu thật.

Kết nối với RL: Discriminator của GAN có thể được tái sử dụng trực tiếp làm **reward signal** cho RL. Nếu discriminator học được “mẫu này trông như thật không”, ta có thể dùng $\log D(x)$ làm reward để dạy agent thực hiện hành vi trông tự nhiên – đây là một trong những cách phá vỡ “abstraction barrier” giữa generative model và RL algorithm mà tutorial nhấn mạnh.

2.2.4. Diffusion Models

Diffusion model là họ generative model hiện đại nhất, đứng sau các hệ thống tạo ảnh mạnh nhất hiện nay như Stable Diffusion và DALL-E. Ý tưởng cốt lõi: **học cách đảo ngược một quá trình phá hủy**.

Quá trình forward (thêm nhiễu). Bắt đầu từ dữ liệu sạch x_0 , ta dần dần thêm Gaussian noise qua T bước để thu được chuỗi $x_1, x_2, \dots, x_T$, trong đó x_T gần với nhiễu trắng hoàn toàn:

$$q(x_t | x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t \mathbf{I})$$

Nhờ tính chất của Gaussian, ta có thể tính thẳng x_t từ x_0 :

$$q(x_t | x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t) \mathbf{I}), \quad \bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s)$$

Quá trình reverse (khử nhiễu). Mô hình học cách đảo ngược quá trình trên – từ x_T nhiễu, dần dần tái tạo dữ liệu sạch x_0 qua nhiều bước. Đây là bước khó vì cần ước

lượng $p(x_{t-1} | x_t)$. Kỹ thuật mẫu chốt là học **score function**:

$$s_\theta(x_t, t) \approx \nabla_{x_t} \log p_t(x_t)$$

Score function $\nabla_{x_t} \log p_t(x_t)$ là gradient của log-probability theo x_t – nó chỉ hướng “đi về phía vùng mật độ xác suất cao hơn” trong không gian dữ liệu tại bước thời gian t .

Kết nối với RL: Score function của diffusion model $\nabla_x \log p_t(x)$ và score function của policy gradient $\nabla_\theta \log \pi_\theta(a | s)$ là cùng một cấu trúc toán học – gradient của log-probability theo biến cần tối ưu. Đây là lý do diffusion model và policy gradient có thể được tích hợp tự nhiên vào nhau, như được minh họa qua DDPO và Diffusion-DICE ở Phần III.

Trong RL, diffusion model được dùng làm **diffusion policy**: thay vì policy Gaussian đơn giản chỉ biểu diễn một mode, agent dùng một diffusion model để biểu diễn phân phối hành động phức tạp, đa mode – quan trọng khi dataset chứa nhiều chiến lược khác nhau.

2.2.5. Flow Matching Models

Flow matching là một hướng tiếp cận thay thế cho diffusion, học cách biến đổi trực tiếp một phân phối đơn giản (thường là Gaussian) thành phân phối dữ liệu thông qua một **vector field** $v_\theta(x, t)$ có thể học được. Quá trình biến đổi được mô tả bởi phương trình vi phân thường (ODE):

$$\frac{dx}{dt} = v_\theta(x, t), \quad x(0) \sim \mathcal{N}(0, I), \quad x(1) \sim p_{\text{data}} \quad (12)$$

Ý tưởng trực quan: ta “vẽ” một đường đi trơn trong không gian xác suất, từ điểm xuất phát (nhiều Gaussian) đến điểm đích (dữ liệu thật). Vector field v_θ chỉ hướng và tốc độ di chuyển tại mỗi điểm.

Ưu điểm so với diffusion. Diffusion model dùng schedule nhiều cố định với hàng trăm bước nhỏ. Flow matching học đường đi **thẳng hơn** trong không gian xác suất, cho phép sinh mẫu với ít bước hơn (đôi khi chỉ 1 bước ODE). Về lý thuyết, flow matching tối ưu hóa trực tiếp likelihood của trajectory thay vì phải thông qua các bước gián tiếp.

Kết nối với RL: Trong tutorial, Temporal Difference Flows [8] áp dụng flow matching để ước lượng *likelihood của trạng thái tương lai* – tức successor measure – kết hợp với cấu trúc Bellman để tận dụng dữ liệu off-policy. Đây là ví dụ điển hình về việc khai thác đặc thù của một generative model family để giải quyết bài toán RL.

2.2.6. Transformer và Autoregressive Models

Transformer là kiến trúc mạng neural dựa trên cơ chế *self-attention*, hiện đang chi phối phần lớn AI hiện đại (GPT, BERT, ViT). Trước khi giải thích ứng dụng trong RL, cần hiểu hai khái niệm cốt lõi:

Self-attention. Với một chuỗi token $(x_1, \dots, x_n)$, self-attention tính **mức độ liên quan** giữa mỗi cặp token và tổng hợp thông tin theo mức độ đó:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

trong đó Q (query), K (key), V (value) là các biến đổi tuyến tính của input. Cơ chế này cho phép mỗi token “chú ý” đến những token liên quan nhất, bất kể chúng cách nhau bao xa trong chuỗi – điều mà RNN và LSTM gặp khó khăn.

Mô hình autoregressive. Mô hình autoregressive sinh chuỗi token từng bước một, trong đó mỗi token được sinh có điều kiện trên toàn bộ các token đã sinh trước đó:

$$p(x_1, x_2, \dots, x_T) = \prod_{t=1}^T p(x_t \mid x_1, \dots, x_{t-1})$$

Đây là nguyên tắc hoạt động của GPT: sinh token tiếp theo dựa trên tất cả token trước đó.

Decision Transformer: autoregressive model làm policy. Decision Transformer [3] áp dụng trực tiếp kiến trúc này cho RL: thay vì học value function hay thực hiện Bellman backup, mô hình coi một trajectory RL – chuỗi $(R_0, s_0, a_0, R_1, s_1, a_1, \dots)$ – như một chuỗi token và học sinh hành động có điều kiện trên *return-to-go* (tổng phần thưởng kỳ vọng từ bước hiện tại trở đi) và lịch sử trajectory.

Ý tưởng then chốt: nếu ta conditioned model trên “return mục tiêu cao” (ví dụ: 100 điểm), model sẽ sinh ra những hành động từng được quan sát trong các trajectory có return cao. Đây là ví dụ điển hình của việc dùng generative model làm policy trong RL.

2.3. Successor Measure

Successor measure (hay discounted state occupancy measure) $p^\pi(s_f)$ là một trong những khái niệm xuyên suốt toàn bộ tutorial. Nó được định nghĩa là xác suất

mà agent đến được trạng thái s_f tại một thời điểm nào đó trong tương lai khi theo policy π :

$$p^\pi(s_f) \triangleq (1 - \gamma) \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t p(s_{t+1} = s_f \mid s_t, a_t) \right] \quad (13)$$

Hệ số $(1 - \gamma)$ chuẩn hóa để p^π là một phân phối xác suất hợp lệ (tổng bằng 1). Phiên bản có điều kiện $p^\pi(s_f \mid s, a)$ cho biết xác suất đến s_f khi bắt đầu từ trạng thái s và thực hiện hành động a .

2.3.1. Ý nghĩa trực quan và ví dụ minh họa

Successor measure trả lời câu hỏi: “Nếu agent bắt đầu từ trạng thái s và theo policy π , nó có bao nhiêu khả năng ghé thăm trạng thái s_f trong tương lai?” Khái niệm này **tổng quát hóa Q-function**: trong khi Q-function tính *một số* (tổng phần thưởng kỳ vọng), successor measure tính *một phân phối đầy đủ* trên các trạng thái tương lai.

Ví dụ minh họa bằng mê cung 3×3 . Hãy tưởng tượng robot đứng tại ô $(1, 1)$ trong mê cung 3×3 , với policy “luôn đi thẳng về phía đông”. Successor measure $p^\pi(s_f \mid (1, 1), \text{đi đông})$ với $\gamma = 0.9$ sẽ trông như sau (giá trị xấp xỉ):

ô $(1, 1)$	ô $(1, 2)$: 0.36 (sau 1 bước: $0.9^0 \cdot 0.4$)	ô $(1, 3)$: 0.32 (sau 2 bước)
... xác suất thấp dần ở các ô phía sau ...		

Khác với Q-function chỉ cho biết “tổng reward kỳ vọng là bao nhiêu”, successor measure cho biết *agent sẽ ở đâu* – thông tin phong phú hơn. Lợi ích then chốt: nếu reward thay đổi (ví dụ: bây giờ ô $(1, 3)$ có thưởng thay vì ô $(1, 2)$), ta chỉ cần tính lại tích phân trên successor measure đã học, **không cần train lại từ đầu**.

2.3.2. Liên hệ với Q-function

Kết nối giữa successor measure và Q-function rất trực tiếp. Nếu có successor measure $p^\pi(s_f \mid s, a)$ và tập trạng thái có nhân thưởng $r(s_f)$, ta có thể tính Q-function bằng:

$$Q^\pi(s, a) = \mathbb{E}_{s_f \sim p^\pi(\cdot \mid s, a)} [r(s_f)] = \int r(s_f) p^\pi(s_f \mid s, a) ds_f \quad (14)$$

Điều này có ý nghĩa lớn: nếu học được successor measure một lần, ta có thể tính Q-function cho **bất kỳ reward function nào** mà không cần huấn luyện lại – đây là nền tảng cho zero-shot RL được trình bày trong Phần III.

2.3.3. Tính toán qua Bellman

Successor measure thỏa mãn một phương trình Bellman tương tự Q-function:

$$p^\pi(s_f | s, a) = (1 - \gamma) p(s' = s_f | s, a) + \gamma \mathbb{E}_{s' \sim p(\cdot | s, a), a' \sim \pi(\cdot | s')} [p^\pi(s_f | s', a')] \quad (15)$$

Diễn giải: xác suất đến s_f từ (s, a) bằng xác suất đến s_f *ngay bước tiếp theo* (số hạng đầu) cộng với xác suất đến s_f *sau nhiều bước* thông qua trạng thái trung gian s' (số hạng sau).

Tính chất này cho phép ước lượng successor measure từ dữ liệu off-policy bằng Temporal Difference learning, tương tự như cách ta ước lượng Q-function. Đây là lý do successor measure có thể được học hiệu quả từ dữ liệu tương tác.

2.3.4. Vị trí trong Tutorial

Successor measure xuất hiện ở nhiều phần khác nhau của tutorial với các vai trò đa dạng. Trong Phần II, nó mô tả phân phối trạng thái đích của goal-reaching problem. Trong Phần III, nó là likelihood của “interactive generative model” cần được ước lượng. Trong zero-shot RL, các biểu diễn forward-backward học cách phân tích successor measure để cho phép tính policy tối ưu cho bất kỳ reward function nào chỉ trong vài bước tính toán.

2.4. Bảng đối chiếu RL và Generative Modeling

Một trong những đóng góp quan trọng nhất của tutorial là chỉ ra sự tương đương toán học giữa RL và Generative Modeling. Bảng 3 tổng hợp các khái niệm tương ứng giữa hai lĩnh vực. Mỗi hàng trong bảng thể hiện một cặp khái niệm đóng *cùng một vai trò toán học* trong hai lĩnh vực khác nhau – hiểu bảng này giúp giải thích tại sao kỹ thuật từ lĩnh vực này có thể áp dụng cho lĩnh vực kia.

Một số cặp chưa quen thuộc sẽ được giải thích trong Phần III: *occupancy measure* $d^\pi(s, a)$ là phân phối tần suất state-action mà policy ghé thăm (tổng quát hóa successor measure sang không gian state-action); *likelihood ratio* là tỉ số $p^\pi(s_f | s, a)/p(s_f)$ đo mức độ policy tăng xác suất đến s_f so với baseline.

Bảng 3: Các khái niệm tương đương giữa RL và Generative Modeling.

Reinforcement Learning	Generative Modeling	Ý nghĩa chung
Policy $\pi(a s)$	Generative model $p(x)$	Phân phối cần học
Score function $\nabla_{\theta} \log \pi(a s)$	Score $\nabla_x \log p(x)$	Gradient của log-xác suất
Return $R(\tau)$	Log-likelihood $\log p(x)$	Tín hiệu học
Trajectory τ	Latent variable z	Biến tiềm ẩn
Policy gradient	Score function estimator	Cách tối ưu phân phối
Q-function $Q(s, a)$	Likelihood ratio $\frac{p^{\pi}(s_f s, a)}{p(s_f)}$	Ước lượng xác suất tương lai
Occupancy measure $d^{\pi}(s, a)$	Data distribution $p(x)$	Phân phối muốn matching
Bellman equation	Markov property	Cấu trúc đệ quy
Skill label z	Latent code z trong VAE	Điều khiển hành vi

Hiểu bảng này giúp đọc phần nội dung chính dễ hơn đáng kể: mỗi khi tutorial dùng kỹ thuật từ một lĩnh vực để giải quyết bài toán của lĩnh vực kia, đều có thể truy về một cặp tương đương trong bảng 3.

3. Nội dung chính của tutorial

Trước khi đi vào từng kỹ thuật, các công trình chính trong phần này được định vị theo vai trò mà chúng đảm nhiệm trong mạch lập luận của tutorial. Mỗi công trình không được trình bày như một mục literature review riêng, mà được dùng để giải quyết một nút thắt kỹ thuật cụ thể: học dynamics, biểu diễn policy, kiểm soát error exploitation, fine-tune diffusion model bằng reward, hoặc chuyển interaction thành distribution matching.

Công trình / vai trò	Kỹ thuật chính	Điểm cần rút ra
PILCO, PETS, Dreamer, Diffuser World model trong model-based RL	Học dynamics bằng conditional likelihood hoặc trajectory model	Giảm chi phí tương tác, nhưng chịu rủi ro rollout error và model exploitation
Decision Transformer Sequence model as policy	Conditioning policy trên return-to-go và trajectory history	Minh họa generative policy trong offline RL, nhưng dễ học correlation thay vì causal effect
Diffusion-DICE Diffusion policy trong offline RL	Guide-then-select với density-ratio guidance và chọn candidate bằng Q	Khai thác biểu diễn đa mode của diffusion mà giảm error exploitation
DDPO RL fine-tuning cho diffusion model	Xem denoising chain như finite-horizon MDP và tối ưu bằng policy gradient	Cho thấy chiều ngược lại: RL có thể tối ưu chính generative model theo downstream reward
GCBC, Dual RL, DICE, f-Policy Gradients Interaction as a generative process	Goal conditioning, occupancy matching, density ratio và f -divergence gradient	Thay reward engineering bằng distribution objective, nhưng vẫn phụ thuộc support và estimation stability

3.1. Generative Models in RL Engines

3.1.1. SL Objective vs RL Objective

Điểm xuất phát của phân tích là sự khác biệt giữa cách supervised learning và reinforcement learning tạo ra dữ liệu. Trong supervised learning, dataset được xem như cố định. Nếu học một policy bằng imitation learning, dữ liệu gồm các cặp trạng thái–hành động (s, a) từ chuyên gia hoặc từ một hệ thống có sẵn, và mô hình được tối

ưu để tăng likelihood của hành động trong dataset:

$$\max_{\theta} J_{\text{SL}}(\theta) = \mathbb{E}_{(s,a) \sim \mathcal{D}} [\log \pi_{\theta}(a \mid s)]. \quad (16)$$

Objective này có bản chất là tái tạo hành vi quan sát được. Trong trường hợp policy Gaussian với phương sai cố định, maximum likelihood tương đương với việc giảm sai số bình phương giữa hành động dự đoán và hành động trong dataset. Vì vậy, chất lượng policy phụ thuộc trực tiếp vào chất lượng dữ liệu. Nếu dataset đến từ một chính sách yếu hoặc không bao phủ những hành vi tốt, supervised objective không tự tạo ra cơ chế để vượt ra khỏi giới hạn đó.

Trong RL, mô hình không chỉ học từ dữ liệu có sẵn mà còn quyết định dữ liệu nào sẽ được sinh ra. Bài toán thường được hình thức hóa bằng một Markov Decision Process gồm state s_t , action a_t , dynamics $p(s_{t+1} \mid s_t, a_t)$, reward $r(s_t, a_t)$ và discount factor γ . Một trajectory có dạng $\tau = (s_0, a_0, s_1, a_1, \dots)$, với return chiết khấu:

$$R(\tau) = \sum_{t=0}^{\infty} \gamma^t r(s_t, a_t). \quad (17)$$

Objective của RL đánh giá policy theo expected discounted return qua các trajectory mà chính policy sinh ra:

$$J_{\text{RL}}(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} [R(\tau)]. \quad (18)$$

Khác biệt then chốt nằm ở phân phối lấy kỳ vọng. Trong Eq. (16), dữ liệu đến từ $\mathcal{D}$ và không phụ thuộc vào tham số policy đang học. Trong Eq. (18), trajectory lại được sinh bởi chính π_{θ} khi policy tương tác với environment. Khi θ thay đổi, phân phối hành động thay đổi, phân phối trạng thái được ghé thăm cũng thay đổi, và do đó phân phối dữ liệu huấn luyện trong tương lai cũng thay đổi.

Do phân phối trajectory phụ thuộc vào policy, tối ưu RL đồng thời thay đổi quy tắc ra quyết định và phân phối dữ liệu mà quy tắc đó sẽ gặp trong tương lai. Khác với supervised learning trên phân phối cố định, sai lệch của policy vì thế có thể làm thay đổi chính miền trạng thái dùng để đánh giá và cập nhật policy. Đặc điểm phân phối này tạo ra mối liên hệ trực tiếp với generative modeling, nhưng bổ sung một ràng buộc riêng của RL: phân phối sinh ra phải được đánh giá thông qua reward và dynamics của môi trường.

3.1.2. Generative Models as World Models

Vai trò trực tiếp nhất của generative models trong RL là world model. Trong nhiều bài toán như robotics, điều khiển hệ thống vật lý, thiết kế phân tử hoặc tương tác với người dùng thật, thu thập dữ liệu bằng cách chạy policy trong môi trường có

thể chậm, tốn kém hoặc nguy hiểm. Model-based RL giải quyết khó khăn này bằng cách học một mô hình dynamics:

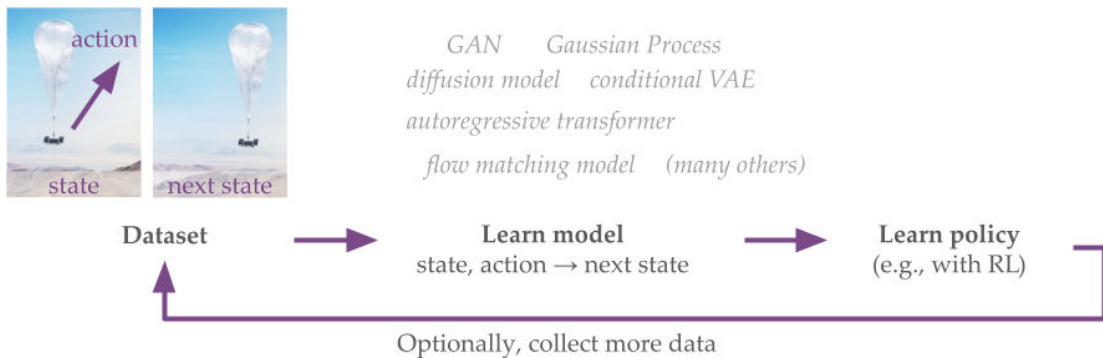
$$p_\phi(s' | s, a) \approx p(s' | s, a). \quad (19)$$

Nếu có dataset chuyển trạng thái $\mathcal{D} = \{(s, a, s')\}$, world model có thể được học bằng maximum likelihood trên các tuple (s, a, s') :

$$\max_{\phi} \mathbb{E}_{(s,a,s') \sim \mathcal{D}} [\log p_\phi(s' | s, a)]. \quad (20)$$

Về mặt huấn luyện, đây là một bài toán conditional generative modeling: từ (s, a) , model sinh hoặc ước lượng distribution của state kế tiếp. Sau khi học được, world model có thể được dùng để rollout, đánh giá policy hoặc huấn luyện policy mà không cần tương tác trực tiếp với environment thật ở mọi bước.

Các công trình model-based RL được đặt trong phần này như các ví dụ về những generative model families có thể thay vào cùng formulation, thay vì là đối tượng khảo sát thuật toán chi tiết. Các lựa chọn tiêu biểu gồm Gaussian process trong **PILCO**, ensemble Gaussian MLP trong **PETS**, latent state-space model trong **Dreamer**, diffusion model trên trajectory trong **Diffuser**, sequence model trong **Decision Transformer** và **Trajectory Transformer**, cùng flow-matching model trong **Temporal Difference Flows** [6, 5, 9, 10, 3, 11, 8]. Danh sách này cho thấy “world model” không chỉ định một architecture cụ thể: model family được lựa chọn quyết định cách biểu diễn uncertainty, high-dimensional observation, multimodality và độ dài của rollout, trong khi vai trò chung vẫn là xấp xỉ dynamics để hỗ trợ planning hoặc policy learning.



Hình 2: Pipeline model-based RL với world model: generative model được huấn luyện để học dynamics từ transition data, sau đó hỗ trợ học policy. Nguồn hình: Figure 3 trong [7].

World model đem lại lợi ích rõ ràng về chi phí tính toán: rollout trong neural model có thể chạy theo batch trên GPU, trong khi simulator thật hoặc hệ thống vật lý

thật có thể rất đắt. Lợi ích tính toán này chỉ có ý nghĩa khi sai số mô hình được kiểm soát trên rollout. Lỗi dự đoán một bước có thể tích lũy thành sai lệch lớn sau nhiều bước, và sai lệch đó còn nguy hiểm hơn khi policy học cách khai thác vùng mà model dự đoán quá lạc quan về reward hoặc dynamics. Do đó, world model không chỉ cần chính xác trung bình trên dataset ban đầu; hệ thống còn cần cơ chế đánh giá độ tin cậy của model và quyết định khi nào phải thu thập thêm dữ liệu từ environment. Hệ quả là độ chính xác one-step chưa đủ để đánh giá world model; độ tin cậy của model còn phụ thuộc vào sai số tích lũy trên rollout và vào phân phối trạng thái do policy đang tối ưu tạo ra.

3.1.3. Reward-Weighted Learning and the Score Function

Sau world model, câu hỏi nền tảng là dữ liệu trong RL đến từ đâu. Trong supervised learning, dataset là input cố định tồn tại độc lập với model đang học. Ngược lại, trong RL, chính policy quyết định dữ liệu nào sẽ được thu thập; vì vậy, cải thiện policy và tạo dữ liệu mới là hai quá trình phụ thuộc lẫn nhau theo một vòng lặp không thể tách rời. Policy tốt cần trajectory tốt để học, nhưng trajectory tốt lại thường chỉ xuất hiện khi policy đã đủ tốt.

Quan hệ này có thể được hình thức hóa bằng một lower bound kiểu ELBO, trong đó trajectory đóng vai trò latent variable. Gọi $\pi_\theta(\tau)$ là xác suất policy sinh trajectory τ , và $R(\tau) > 0$ là return. Với một variational distribution $q(\tau)$ dương trên các trajectory được xét, ta nhân và chia integrand cho $q(\tau)$:

$$\begin{aligned}\log \mathbb{E}_{\pi_\theta(\tau)}[R(\tau)] &= \log \int \pi_\theta(\tau) R(\tau) d\tau \\ &= \log \int q(\tau) \frac{\pi_\theta(\tau) R(\tau)}{q(\tau)} d\tau.\end{aligned}\tag{21}$$

Phép biến đổi này không làm thay đổi expected return, nhưng đưa biểu thức về log của một kỳ vọng theo q , là dạng cần thiết để áp dụng Jensen. Giả thiết $R(\tau) > 0$ bảo đảm đối số của log dương. Nếu return không dương, phép suy diễn này không áp dụng trực tiếp; cần xác định một hàm trọng số dương từ return và khi đó objective tái trọng số cũng thay đổi tương ứng. Khi đó:

$$\log \mathbb{E}_{\pi_\theta(\tau)}[R(\tau)] \geq \mathbb{E}_{q(\tau)} [\log \pi_\theta(\tau) + \log R(\tau) - \log q(\tau)].\tag{22}$$

Dưới góc nhìn variational inference, trajectory τ đóng vai trò như một latent variable. Đại lượng tương ứng với evidence trong phép diễn giải này là expected return $\mathbb{E}_{\pi_\theta(\tau)}[R(\tau)]$, và Eq. (22) xây dựng một lower bound cho log của đại lượng đó. Các trajectory tạo ra return cao chưa được biết trước và được mô hình hóa thông qua

variational distribution $q(\tau)$.



Hình 3: Diễn giải RL như một bài toán latent-variable, trong đó các path đến high-reward state là latent variables cần được suy luận. Nguồn hình: Figure 6 trong [7].

Trong một bước coordinate update, ký hiệu $\pi_{\theta_{\text{old}}}$ là policy đã sinh batch trajectory hiện tại. Tối ưu lower bound theo q với policy này được giữ cố định cho nghiệm:

$$q^*(\tau) = \frac{R(\tau)\pi_{\theta_{\text{old}}}(\tau)}{\int R(\tau')\pi_{\theta_{\text{old}}}(\tau')d\tau'}. \quad (23)$$

Phân phối q^* tái trọng số trajectory của policy cũ theo return và không tạo xác suất trên các trajectory nằm ngoài support đã được lấy mẫu. Vì vậy, policy update chỉ có thể nhấn mạnh các hành vi đã xuất hiện; khả năng khám phá vẫn quyết định những vùng trajectory nào có thể được tối ưu ở bước kế tiếp.

Khi giữ q^* cố định và tối ưu policy mới, các hằng số chuẩn hóa không phụ thuộc vào θ có thể được bỏ qua. Sau khi giữ q^* cố định, reward-weighted regression tối ưu likelihood của trajectory dưới trọng số return:

$$\mathcal{L}_{\text{RWR}}(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta_{\text{old}}}} [R(\tau) \log \pi_{\theta}(\tau)]. \quad (24)$$

Do trajectory likelihood phân rã theo các action, gradient của objective này có dạng:

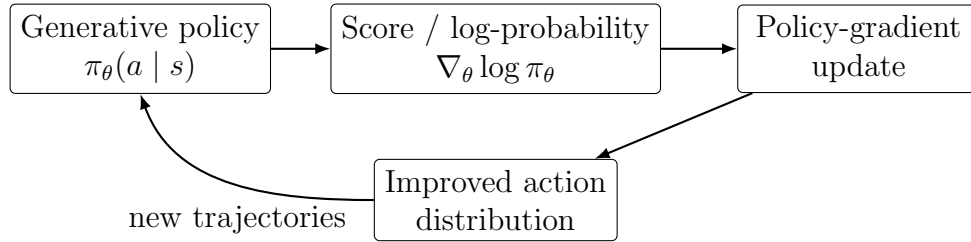
$$\nabla_{\theta} \mathcal{L}_{\text{RWR}}(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta_{\text{old}}}} \left[R(\tau) \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right]. \quad (25)$$

Kỳ vọng trong Eq. (25) được lấy theo policy ở vòng trước, còn log-likelihood được tối ưu theo tham số policy mới. Đây là cấu trúc update trong phép suy diễn ELBO của tutorial: lấy mẫu trajectory, cố định batch, gán trọng số bằng return và thực hiện maximum likelihood có trọng số [15].

Cần phân biệt estimator trên với policy gradient on-policy của expected return. Nếu lấy gradient trực tiếp của expected return thay vì tối ưu lower bound, ta thu được policy gradient estimator:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[R(\tau) \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right]. \quad (26)$$

Số hạng $\nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$ chính là score function của policy. Sự xuất hiện của cùng score function trong Eq. (25) và Eq. (26) là cầu nối kỹ thuật giữa RL và generative modeling: bất kỳ generative model family nào cho phép tính hoặc ước lượng score của action đều có thể được tích hợp vào cập nhật kiểu policy gradient [18]. Kết quả này không phụ thuộc vào giả thiết policy Gaussian; transformer, diffusion model và flow model khác nhau ở cách tham số hóa phân phối và xây dựng score estimator.



Hình 4: Sơ đồ cầu nối giữa generative policy, score function và cập nhật policy bằng policy gradient.

3.1.4. Generative Models as Policies

Khi policy được nhìn như phân phối có điều kiện $\pi_{\theta}(a | s)$, chính policy đã là một generative model sinh action từ state. Ở mức kỹ thuật, các policy distribution đơn giản được thay bằng những kiến trúc sinh mạnh hơn, đặc biệt là sequence models và diffusion models.

Lecture notes trước hết trừu tượng hóa hướng sequence modeling cho RL bằng cách xem offline trajectory như một chuỗi state, action và reward [7]:

$$S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1}, \dots$$

Trong công trình **Decision Transformer**, ý tưởng này được cụ thể hóa bằng một autoregressive transformer không học value function hoặc Bellman backup trực tiếp, mà học phân phối hành động có điều kiện trên return-to-go và lịch sử trajectory [3]: $\pi(a_t | s_{\leq t}, a_{< t}, R_{\text{target}})$, thay cho conditioning chỉ trên state hiện tại. Giá trị return mục tiêu đóng vai trò biến điều kiện điều chỉnh phân phối hành động học được từ offline data. Điểm mạnh của cách tiếp cận này là tận dụng năng lực sequence modeling đã thành công trong generative AI. Hạn chế là mô hình chủ yếu học correlation trong offline trajectories. Nếu một trajectory có return cao do may mắn trong môi trường stochastic, việc condition trên return cao có thể khiến mô hình bắt chước action không thực sự là nguyên nhân tạo ra reward cao [14]. Vì vậy, vai trò của **Decision Transformer** trong phần này không phải là thay thế toàn bộ RL bằng language modeling, mà là minh họa một policy class mới: policy được mô hình hóa như conditional sequence model. Nút thắt kỹ thuật mà công trình này làm rõ là sự khác biệt giữa biểu diễn một phân

phối hành động phức tạp và bảo đảm policy improvement khi dữ liệu chỉ đến từ offline trajectories.

Diffusion policy mở rộng ý tưởng generative policy sang phân phối hành động đa mode. Trong offline RL, cùng một state có thể có nhiều action hợp lệ do dataset chứa nhiều chiến lược khác nhau. Một Gaussian policy đơn mode có thể trung bình hóa các mode và sinh action không thuộc mode nào. Ngược lại, diffusion model có thể sinh action qua nhiều bước denoising và biểu diễn phân phối phức tạp hơn. Tuy nhiên, expressivity cao không tự bảo đảm policy improvement: khi diffusion policy được kết hợp với critic trong offline RL, các candidate ngoài support dữ liệu có thể nhận giá trị dự đoán sai. **Diffusion-DICE** tập trung vào chính tương tác giữa khả năng sinh đa mode và sai số ngoại suy của critic.

3.1.5. Case Study: Diffusion-DICE

Diffusion-DICE là case study trung tâm cho diffusion policy trong offline RL [12]. Vấn đề kỹ thuật của phương pháp là tận dụng khả năng sinh action đa mode của diffusion model mà không để critic kéo policy ra khỏi support của dữ liệu. Trong diffusion policy, action sạch a_0 được sinh từ chuỗi denoising $a_T, \dots, a_0$ có điều kiện trên state s . Mục tiêu không chỉ là bắt chước behavior policy trong dataset, mà là sinh action có giá trị cao theo reward hoặc Q-function. Nút thắt mà công trình này xử lý là một điểm yếu cụ thể của diffusion policy trong offline RL: model đủ mạnh để sinh nhiều candidate actions, nhưng critic học từ offline data có thể đánh giá sai các candidate nằm ngoài support. Do đó, kỹ thuật cốt lõi không chỉ là thêm guidance vào diffusion sampling, mà là kiểm soát cách guidance được dùng để tránh error exploitation.

Hai hướng tiếp cận chính là guide-based và select-based. Guide-based methods học diffusion behavior policy rồi thêm guidance term để đẩy quá trình sampling về vùng có giá trị cao:

$$\nabla_{a_t} \log \pi_t(a_t | s) = \nabla_{a_t} \log \pi_t^D(a_t | s) + \nabla_{a_t} J_t(a_t, s). \quad (27)$$

Ở đây π_t^D là behavior diffusion policy, còn J_t là tín hiệu hướng dẫn học từ critic hoặc objective liên quan đến reward. Đối với nhóm select-only, một cách triển khai điển hình là sinh nhiều candidate actions từ behavior policy rồi tái lấy mẫu theo trọng số do critic cung cấp:

$$\Pr[a = a_0^j | s] = \frac{w(s, a_0^j)}{\sum_{i=1}^N w(s, a_0^i)}. \quad (28)$$

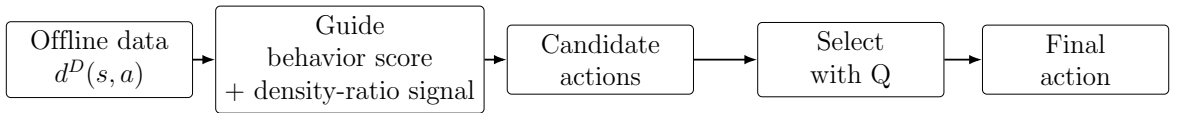
Eq. (28) mô tả cơ chế resampling của nhóm baseline select-based, chẳng hạn **IDQL**; đây chưa phải bước Select cuối cùng của Diffusion-DICE [?]. Cả hai hướng đều phụ

thuộc vào critic. Trong offline RL, critic chỉ được học từ dataset cố định; vì vậy critic có thể overestimate các action ngoài phân phối. Nếu sampling hoặc guidance đưa action ra khỏi vùng dữ liệu, policy có thể khai thác lỗi critic thay vì tìm action thật sự tốt. Hiện tượng này được gọi là error exploitation.

Diffusion-DICE đề xuất paradigm guide-then-select. Bước guide dùng score của behavior policy cộng với guidance liên quan đến optimal distribution ratio để sinh candidate actions. Điểm cần trình bày cẩn thận là guidance term này không được tính bằng một closed-form đơn giản trong thực tế. Theo Diffusion-DICE, thành phần liên quan đến density ratio thường intractable vì đòi hỏi các kỳ vọng điều kiện; phương pháp biến đổi bài toán thành objective có thể học in-sample. Sau đó, Diffusion-DICE thực hiện lựa chọn cứng trong tập K candidate actions được sinh từ optimal policy distribution $\pi^*(a | s)$ mà bước Guide xấp xỉ. Đặt $\mathcal{C}_K(s) = \{a_0^1, \dots, a_0^K\}$ với $a_0^k \sim \pi^*(\cdot | s)$, bước Select được viết:

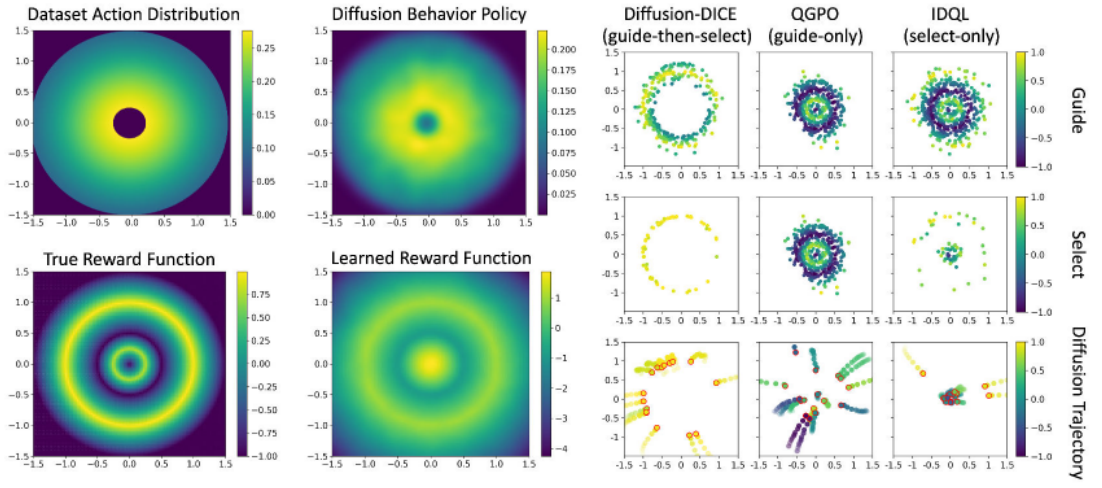
$$\pi(s) = a^* \triangleq \arg \max_{a \in \mathcal{C}_K(s)} Q^*(s, a). \quad (29)$$

Khác với phép resampling trong Eq. (28), Eq. (29) chọn candidate có giá trị Q lớn nhất, thay vì gán xác suất lựa chọn tỉ lệ với trọng số [12]. Nhờ vậy, critic vẫn được dùng để chọn action tốt hơn, nhưng quá trình lựa chọn được giữ gần vùng dữ liệu hơn.



Hình 5: Luồng guide-then-select trong Diffusion-DICE: bước Guide cộng behavior score với guidance liên quan đến density ratio để tạo candidate actions, còn bước Select dùng Q-function trong tập candidate để giảm khai thác lỗi critic ngoài phân phối dữ liệu.

Toycase 2D bandit cô lập chính xác vấn đề này. Offline dataset chỉ bao phủ một vùng hình khuyên (annular region). True reward cao ở outer circle, nhưng learned critic lại overestimate một số vùng inner circle do thiếu dữ liệu. Guide-only có thể bị kéo mạnh vào vùng critic overestimate, còn select-only bị giới hạn bởi chất lượng candidate. Diffusion-DICE giảm rủi ro bằng cách kết hợp hai bước: guide để tạo candidate hướng tới phân phối tối ưu, và select để chọn trong tập candidate theo hướng in-sample. **QGPO** và **IDQL** có thể được xem như các baseline đại diện cho hai cực guide-only và select-only, nhưng trọng tâm kỹ thuật của phần này là cơ chế guide-then-select của Diffusion-DICE [?, ?].



Hình 6: Toy case 2D bandit của Diffusion-DICE: dataset action nằm trong vùng hình khuyên; true reward cao ở outer circle; learned reward overestimate inner circle; Diffusion-DICE giảm error exploitation tốt hơn các hướng guide-only hoặc select-only. Nguồn hình: Figure 7 trong [7].

3.1.6. DDPO: Viewing Denoising as an MDP

Nếu Diffusion-DICE dùng diffusion model để xây policy cho RL, **DDPO** đi theo chiều ngược lại: dùng RL để fine-tune diffusion model theo downstream reward [1]. Động lực của DDPO là nhiều mục tiêu quan trọng trong image generative models không được mô tả tốt bằng maximum likelihood. Aesthetic quality, prompt-image alignment hoặc compressibility là các tiêu chí được đánh giá ở sample cuối, không phải likelihood của từng bước huấn luyện diffusion. Do đó, fine-tuning bằng reward trở thành một bài toán RL tự nhiên. Ba thành phần kỹ thuật xác định **DDPO**: reward chỉ xuất hiện ở final sample, chuỗi denoising cung cấp một dãy decision steps, và policy gradient phân bổ tín hiệu reward cuối về từng denoising transition. Nhờ đó, DDPO trở thành một formulation RL đầy đủ cho sampling process của diffusion model, thay vì chỉ là một thủ thuật tái trọng số các sample cuối có reward cao.

Trong diffusion sampling, một sample cuối x_0 được tạo ra từ chuỗi denoising:

$$x_T \rightarrow x_{T-1} \rightarrow \dots \rightarrow x_0.$$

DDPO mô hình hóa chuỗi này như một finite-horizon MDP. Với prompt hoặc context c , state tại timestep t là noisy sample x_t . Action không nên hiểu cứng là predicted noise; cách mô tả đúng hơn là denoising decision, tức lấy mẫu x_{t-1} từ policy:

$$p_\theta(x_{t-1} \mid x_t, c). \quad (30)$$

Reward được chấm ở sample cuối, ký hiệu chung là $r(x_0, c)$; các ví dụ được DDPO khảo

sát gồm compressibility, aesthetic quality và prompt–image alignment [1]. Theo bài báo DDPO, khi đó objective là kỳ vọng reward trên denoising trajectory, và policy-gradient estimator có dạng:

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[r(x_0, c) \sum_{t=1}^T \nabla_{\theta} \log p_{\theta}(x_{t-1} \mid x_t, c) \right]. \quad (31)$$

Công thức này cho thấy reward cuối được gán credit cho toàn bộ chuỗi denoising transitions. So với reward-weighted regression, DDPO giữ lại cấu trúc nhiều bước của quá trình sampling. RWR chỉ tăng likelihood của sample cuối có reward cao, trong khi DDPO cập nhật log-probability của từng transition $x_t \rightarrow x_{t-1}$. Theo bài báo DDPO, cách tiếp cận này hiệu quả hơn RWR trong các objective được khảo sát vì RWR gặp các vấn đề optimization và approximation trong diffusion setting [1].

Ở mức cụ thể hóa trong bài báo gốc, formulation này dẫn tới hai estimator. **DDPO-SF** áp dụng trực tiếp score-function estimator theo tinh thần REINFORCE trên các denoising trajectories. **DDPO-IS** bổ sung importance sampling để tái sử dụng trajectory đã thu thập cho nhiều bước cập nhật hơn, đồng thời dùng clipping theo tinh thần PPO nhằm kiểm soát policy shift. Các chi tiết này thuộc phần triển khai của bài báo DDPO; trong tutorial, vai trò chính của DDPO là minh họa việc xem denoising như MDP và tối ưu diffusion model bằng policy gradient [1].

Trong formulation tổng quát của DDPO, chuỗi denoising được xem như một MDP và được tối ưu bằng policy gradient thay cho reward-weighted likelihood. Bài báo gốc cho thấy formulation này có thể fine-tune image generative models theo các downstream criteria như aesthetics, compressibility và prompt–image alignment hiệu quả hơn RWR trong các thiết lập được khảo sát; tuy nhiên, tối ưu một reward proxy không hoàn chỉnh vẫn có thể dẫn tới overoptimization hoặc reward hacking [1, 2].

Trong mạch lập luận chung, DDPO hoàn thiện quan hệ hai chiều giữa hai lĩnh vực. Diffusion-DICE đưa diffusion model vào RL để biểu diễn policy phức tạp; ngược lại, DDPO đưa policy optimization vào diffusion model để điều chỉnh chính sampling process. Cả hai công trình đều sử dụng cấu trúc nhiều bước của diffusion, nhưng tối ưu hai đối tượng khác nhau: Diffusion-DICE tìm action có giá trị cao trong offline RL, còn DDPO tối ưu chất lượng sample cuối theo downstream reward.

3.2. Interaction as a Generative Model

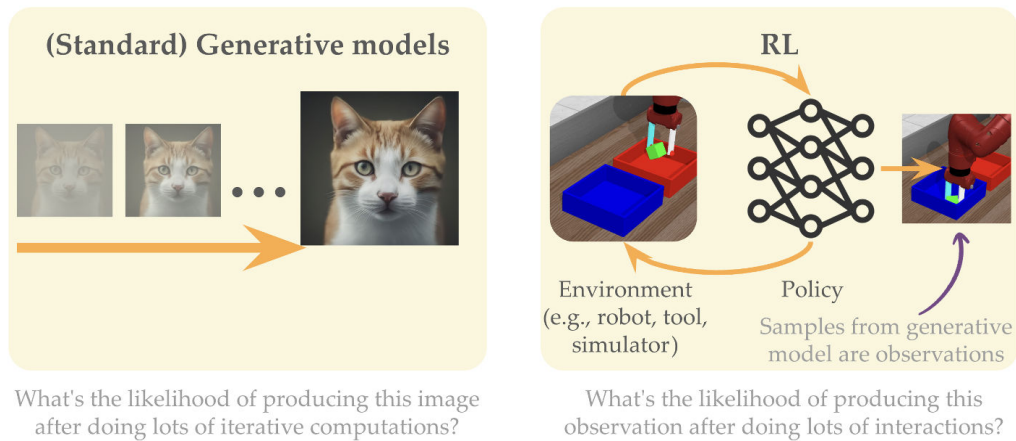
3.2.1. Interaction as a Generative Process

Góc nhìn thứ hai xem toàn bộ quá trình tương tác giữa policy và environment như một generative model. Một diffusion model tạo ảnh bằng nhiều bước denoising. Tương tự, một RL agent tạo observation tương lai bằng nhiều bước tương tác: policy sinh action, environment sinh state kế tiếp, rồi quá trình lặp lại. Nếu s_0 là trạng thái ban đầu, quá trình

$$a_t \sim \pi(a_t | s_t), \quad s_{t+1} \sim p(s_{t+1} | s_t, a_t)$$

xác định một phân phối trên các state hoặc observation tương lai. Trong cách nhìn này, agent-environment system là một generative process, còn sample của nó là trạng thái mà agent đạt được sau nhiều bước interaction.

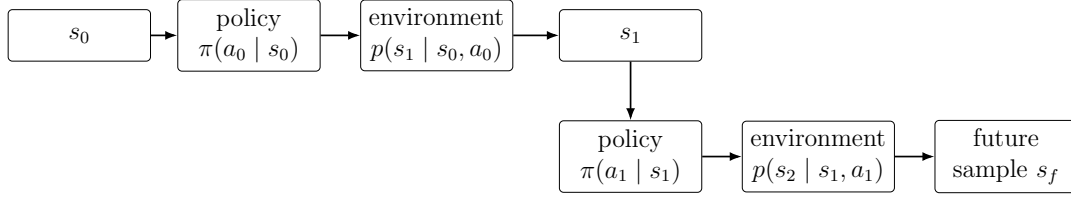
Ba động lực kỹ thuật làm cho cách mô hình hóa này hữu ích trong bối cảnh GenAI và RL. Động lực thứ nhất là RL cung cấp ngôn ngữ hình thức để mô tả hệ thống sinh có tương tác với con người, công cụ hoặc môi trường vật lý, kể cả khi một phần quá trình là hộp đen hoặc không khả vi. Động lực thứ hai là đặc tả tác vụ bằng scalar reward thường đòi hỏi reward engineering và reward shaping thủ công; các thành phần trung gian có thể vô tình mã hóa trước cách giải thay vì chỉ mô tả kết quả mong muốn. Động lực thứ ba là scalar reward tạo ra sự lệch pha với generative modeling thông thường, nơi supervision được cung cấp dưới dạng dữ liệu mẫu. Part II khảo sát khả năng đặc tả một lớp bài toán RL bằng desired observations hoặc goal data, qua đó chuyển một phần bài toán từ thiết kế reward sang khớp phân phối outcome [7].



Hình 7: So sánh generative models chuẩn với RL interaction. Cả hai đều dùng nhiều bước tính toán hoặc tương tác để tạo sample. Nguồn hình: Figure 8 trong [7].

Cách nhìn này không yêu cầu toàn bộ generation chain phải được biểu diễn bởi một mạng khả vi: policy, environment, tool và human feedback chỉ cần cùng xác định một distribution trên outcome. Bài toán điều khiển sau đó có thể được phát biểu dưới

dạng thay đổi policy để tăng likelihood của các outcome mong muốn, với goal-reaching là trường hợp cụ thể được phân tích tiếp theo.



Hình 8: Interaction generative process: observation tương lai được sinh bằng cách luân phiên lấy mẫu từ policy và environment.

3.2.2. Goal-Reaching and Occupancy Measures

Goal-reaching là thiết lập đơn giản nhất cho quan điểm interaction-as-generation. Thay vì thiết kế reward thủ công nhiều thành phần, task được mô tả bằng một observation mong muốn g . Goal-reaching trước hết có thể được hiểu bằng trực giác indicator trong finite-state setting: agent nhận tín hiệu dương khi đạt goal và nhận tín hiệu bằng không ở các trạng thái khác. Để có formulation áp dụng được cho cả observation rời rạc lẫn liên tục, objective chính được viết qua xác suất transition tới goal:

$$\max_{\pi} (1 - \gamma) \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t p(s_{t+1} = g \mid s_t, a_t) \right]. \quad (32)$$

Hệ số $(1 - \gamma)$ chuẩn hóa tổng chiết khấu thành một phân phối. Objective này tương đương một bài toán RL chuẩn với reward được định nghĩa từ xác suất đạt observation mong muốn:

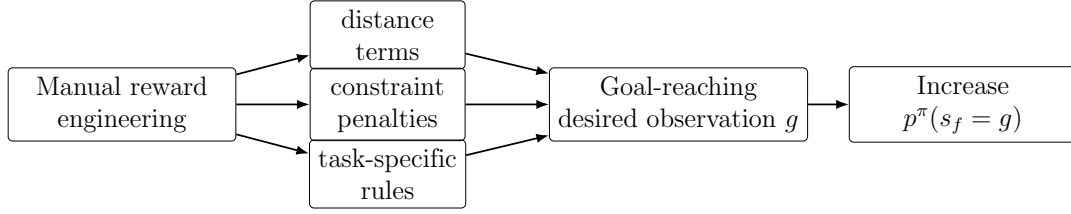
$$r_g(s, a) = (1 - \gamma) p(s' = g \mid s, a). \quad (33)$$

Reward ở đây không phải reward thủ công theo nhiều rule, mà được định nghĩa từ xác suất đạt observation mong muốn. Điều này chuyển reward engineering sang bài toán likelihood của outcome.

Discounted state occupancy measure mô tả phân phối state tương lai do policy sinh ra:

$$p^{\pi}(s_f) \triangleq (1 - \gamma) \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t p(s_{t+1} = s_f \mid s_t, a_t) \right]. \quad (34)$$

Nếu interaction là một generative model, $p^{\pi}(s_f)$ chính là phân phối sample mà model tạo ra. Goal-reaching khi đó trở thành bài toán điều chỉnh policy sao cho phân phối p^{π} đặt xác suất cao lên goal hoặc goal distribution.



Hình 9: Goal-reaching thay nhiều thành phần reward thủ công bằng đặc tả outcome dưới dạng observation mong muốn.

Hạn chế của goal-reaching là không phải mọi tác vụ đều có thể được đặc tả bằng một trạng thái đích đơn lẻ. Chẳng hạn, trạng thái “căn phòng sạch” có thể tương ứng với nhiều cấu hình quan sát hợp lệ, trong khi tiêu chí “chuyến bay êm” phụ thuộc vào các ràng buộc được duy trì trên toàn trajectory. Về mặt kỹ thuật, các ví dụ này đại diện cho tác vụ có nhiều outcome tương đương hoặc có trajectory constraints, do đó không thể quy về một goal observation duy nhất. Do đó, goal-reaching chỉ là một trường hợp đặc biệt của bài toán điều khiển phân phối outcome; các tác vụ tổng quát hơn đòi hỏi formulation trên trajectory distribution hoặc reward function đầy đủ.

3.2.3. Goal-Conditioned Behavioral Cloning

Goal-Conditioned Behavioral Cloning (GCBC) khai thác cấu trúc trajectory để học policy hướng goal từ dữ liệu. Trong hệ thống các công trình của phần Interaction as a Generative Model, GCBC đóng vai trò baseline có tính supervised learning: nó tận dụng trajectory có sẵn bằng cách biến future observation thành goal label, trước khi các phương pháp Dual RL và DICE mở rộng sang occupancy-level distribution matching. Từ một trajectory trong dataset, ta chọn một future state s_f làm goal, rồi huấn luyện policy sinh action hiện tại có điều kiện trên goal:

$$\max_{\theta} \mathbb{E}_{\tau \sim \mathcal{D}} [\mathbb{E}_{(s,a,s_f) \sim \tau} [\log \pi_{\theta}(a \mid s, g = s_f)]] . \quad (35)$$

Bayesian interpretation. Eq. (35) có hình thức của behavioral cloning với biến điều kiện goal, nhưng Bayes rule cho thấy objective còn mã hóa xác suất đạt future state. Action được ưu tiên không chỉ vì nó xuất hiện trong dữ liệu, mà vì trong trajectory gốc nó nằm trên đường dẫn đến future state được chọn làm goal. Một cách viết bỏ qua các hằng số không phụ thuộc action là:

$$\mathbb{E}_{a \sim \pi_{\theta}(a|s,g)} [\log p(s_f = g \mid s, a) + \log \beta(a \mid s)] , \quad (36)$$

trong đó $\beta(a \mid s)$ là behavior policy, còn $p(s_f = g \mid s, a)$ là xác suất đạt future goal khi bắt đầu từ (s, a) và tiếp tục theo policy đang xét. Term đầu khuyến khích action làm

tăng khả năng đạt goal; term thứ hai regularize action về vùng hành vi trong dataset. GCBC do đó kết hợp tín hiệu goal-reaching với regularization về behavior support, thay vì chỉ sao chép marginal action distribution.

Masking interpretation. Diễn giải trọng tâm thứ hai xem GCBC như một bài toán masked sequence modeling trên trajectory. Trong language modeling, masking che một phần token rồi học dự đoán token bị che. Trong trajectory modeling, có thể che action hoặc state rồi yêu cầu mô hình khôi phục phần thiếu. Với GCBC, state hiện tại và goal tương lai đóng vai trò context, còn action là phần cần dự đoán. Nếu mask action, mô hình hoạt động như policy; nếu mask future state, cùng kiến trúc hoạt động như world model. Sự thay đổi vai trò được quyết định bởi biến nào được quan sát và biến nào phải sinh, chứ không nhất thiết bởi một kiến trúc riêng cho từng chức năng.

GCBC cho thấy một cách tận dụng offline trajectories bằng supervised learning, nhưng policy học được vẫn bị ràng buộc mạnh bởi behavior support và các future states đã xuất hiện trong dữ liệu. Để phân tích các phương pháp vượt ra ngoài việc clone trực tiếp hành vi, cần chuyển từ likelihood của action sang phân phối occupancy mà policy tạo ra. Ở mức on-policy, f -divergence gradient cung cấp một cầu nối tự nhiên giữa khớp phân phối goal và policy gradient; trong offline setting, yêu cầu học từ dataset cố định dẫn tới các formulation dual và DICE được trình bày tiếp theo [7].

3.2.4. Dual RL and DICE

Goal-reaching yêu cầu ước lượng xác suất đạt goal, nhưng density estimation trực tiếp trong không gian observation cao chiều thường khó. **Dual RL** giải quyết vấn đề này bằng cách làm việc với occupancy measure và distribution matching. Thay vì tối ưu trực tiếp trên policy, ta xét state-action occupancy $d^\pi(s, a)$ do policy tạo ra. Một objective regularized có dạng:

$$\max_{\pi} \mathbb{E}_{(s,a) \sim d^\pi} [r(s, a)] - \alpha D_f(d^\pi(s, a) \| d^O(s, a)), \quad (37)$$

trong đó d^O là phân phối tham chiếu và D_f là f -divergence. Occupancy distribution không thể tùy ý; nó phải thỏa các Bellman-flow constraints do dynamics quy định. Một dạng ràng buộc Bellman-flow tương ứng với primal-Q trong notes là:

$$d(s, a) = (1 - \gamma)d_0(s)\pi(a | s) + \gamma \sum_{s', a'} d(s', a')p(s | s', a')\pi(a | s), \quad \forall s, a. \quad (38)$$

Ràng buộc này đảm bảo $d(s, a)$ là một occupancy measure hợp lệ, tức nó có thể được sinh ra bởi một policy trong MDP thay vì là một phân phối tùy ý trên state-action

pairs. Bằng Lagrangian duality và convex conjugate, các ràng buộc flow có thể được chuyển thành objective theo các biến dual như $Q(s, a)$ hoặc $V(s)$. Ví dụ, một dạng dual-V được notes trình bày là:

$$\min_V (1 - \gamma) \mathbb{E}_{s \sim d_0}[V(s)] + \alpha \mathbb{E}_{(s,a) \sim d^O} \left[f_p^* \left(\frac{\mathcal{T}V(s, a) - V(s)}{\alpha} \right) \right], \quad (39)$$

trong đó $\mathcal{T}V(s, a)$ là Bellman target của V và f_p^* là biến thể convex conjugate dùng để xử lý ràng buộc không âm của occupancy. Ý nghĩa kỹ thuật là bài toán distribution matching có thể được học bằng loss trên dữ liệu logged, thay vì yêu cầu tương tác on-policy liên tục.

DICE methods là họ phương pháp tiêu biểu cho ý tưởng này. Trong offline setting, DICE xét objective:

$$\pi^* = \arg \max_{\pi} \mathbb{E}_{(s,a) \sim d^{\pi}}[r(s, a)] - \alpha D_f(d^{\pi} \| d^D), \quad (40)$$

trong đó d^D là distribution của offline dataset. Bài toán trực tiếp khó vì d^{π} phụ thuộc vào policy và dynamics. Sau biến đổi dual, objective có thể được ước lượng từ các mẫu (s, a, s') trong dataset. Tính in-sample của DICE nằm ở việc các loss dual được đánh giá trên transition đã ghi nhận, qua đó tránh yêu cầu truy vấn giá trị của toàn bộ không gian action ngoài support dữ liệu. Trong triển khai thực tế, notes trình bày một dạng semi-gradient dùng $Q(s, a)$ để xấp xỉ Bellman target và $V(s)$ để biểu diễn biến dual theo state:

$$\begin{aligned} \min_V \quad & \mathbb{E}_{(s,a) \sim d^D} \left[(1 - \gamma)V(s) + \alpha f^* \left(\frac{Q(s, a) - V(s)}{\alpha} \right) \right], \\ \min_Q \quad & \mathbb{E}_{(s,a,s') \sim d^D} [r(s, a) + \gamma V(s') - Q(s, a)]^2. \end{aligned} \quad (41)$$

Hai loss này chỉ cần các transition đã ghi trong offline dataset, nên phù hợp với mục tiêu in-sample của DICE methods.

Một đại lượng trung tâm là stationary distribution ratio:

$$w^*(s, a) := \frac{d^*(s, a)}{d^D(s, a)}. \quad (42)$$

Tỉ số này cho biết state-action pair trong dataset nên được nhấn mạnh hơn hay ít hơn để tiến đến occupancy tối ưu. Từ góc nhìn tổng hợp của nhóm, density ratio ở đây cũng cung cấp một cách diễn giải cho guidance trong Diffusion-DICE: cả hai đều mô tả sự dịch chuyển từ behavior distribution về phía optimal distribution, trong khi ràng buộc in-sample hạn chế error exploitation. Liên hệ này là phép đối chiếu giữa hai phần của tutorial, không phải một đồng nhất thức được lecture notes phát biểu trực

tiếp [7, 12].

3.2.5. f-Policy Gradients and State-MaxEnt RL

Dual formulation cho biết cách biểu diễn bài toán bằng occupancy measure và density ratio. Từ formulation này, câu hỏi trực tiếp hơn là: nếu goal được mô tả bởi một phân phối trạng thái đích $p_g(s)$, policy cần được cập nhật như thế nào để phân phối trạng thái mà agent tạo ra tiến gần tới phân phối này? Gọi $p_\theta(s)$ là discounted state distribution sinh bởi policy π_θ . Bài toán distribution matching được viết dưới dạng:

$$J(\theta) = D_f(p_\theta(s) \| p_g(s)). \quad (43)$$

Objective này thay việc thiết kế reward riêng cho từng trạng thái bằng yêu cầu khớp phân phối outcome của agent với phân phối goal.

Theorem 3.1 trong lecture notes cung cấp gradient của Eq. (43):

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta(\tau)} \left[\left(\sum_{t=1}^T \nabla_\theta \log \pi_\theta(a_t | s_t) \right) \left(\sum_{t=1}^T f' \left(\frac{p_\theta(s_t)}{p_g(s_t)} \right) \right) \right]. \quad (44)$$

Thừa số thứ nhất là tổng score function của policy trên trajectory. Thừa số thứ hai cung cấp dense weights từ density ratio giữa state distribution hiện tại và goal distribution. Do bài toán tối thiểu hóa $J(\theta)$, hướng cập nhật có cấu trúc giống policy gradient với tín hiệu hiệu dụng $-f'(p_\theta(s_t)/p_g(s_t))$, thay vì reward thừa được thiết kế thủ công. Tuy nhiên, lecture notes lưu ý rằng không thể đồng nhất phép cập nhật này với việc cực đại hóa một surrogate return thông thường, bởi density ratio và trọng số tương ứng cũng phụ thuộc vào θ . Estimator trên còn mang tính on-policy, nên đòi hỏi trajectory từ policy hiện tại và có thể kém hiệu quả dữ liệu [7].

Trường hợp cụ thể với forward KL minh họa rõ hơn mối liên hệ giữa distribution matching và exploration. Khi D_f là $D_{\text{KL}}(p_\theta \| p_g)$, việc cực tiểu hóa divergence tương đương với:

$$\max_{\pi_\theta} \mathbb{E}_{s \sim p_\theta} [\log p_g(s)] + \mathcal{H}(p_\theta(s)). \quad (45)$$

Term thứ nhất đưa state distribution về các vùng có xác suất cao dưới goal distribution; term thứ hai cực đại hóa entropy của chính state occupancy. Đây là điểm khác với MaxEnt RL thông dụng như soft Q-learning hoặc SAC, vốn regularize entropy của action policy. Một policy có action entropy cao vẫn có thể tạo ra state distribution hẹp nếu dynamics khiến nhiều action dẫn tới cùng một vùng trạng thái. Ngược lại,

state-MaxEnt RL tác động trực tiếp lên độ phủ của các trạng thái được ghé thăm. Đặc biệt, nếu p_g là phân phối đều trên state space, term $\log p_g(s)$ trở thành hằng số và objective còn lại là cực đại hóa $\mathcal{H}(p_\theta(s))$, tức khuyến khích agent khám phá rộng trong môi trường. Diễn giải này là nội dung được nhấn mạnh trong phần f-Policy Gradients và state-MaxEnt RL của slide tutorial [7].

3.2.6. Offline Goal-Conditioned RL and Technical Synthesis

Trong offline goal-conditioned RL, goal-transition distribution $q(s, a, g)$ được dùng để mô tả những transition dẫn tới goal. Với một stochastic MDP, notes định nghĩa phân phối này theo dạng:

$$q(s, a, g) \propto q_{\text{train}}(g) \mathbb{E}_{s' \sim p(\cdot | s, a)} [\mathbf{1}\{\phi(s') = g\}]. \quad (46)$$

Bài toán được viết dưới dạng occupancy matching:

$$D_f(d^{\pi_g}(s, a, g) || q(s, a, g)). \quad (47)$$

Vì q có thể không đạt được chính xác bởi bất kỳ policy nào và offline data có support hữu hạn, dual formulation với divergence phù hợp được dùng để thu được objective ổn định hơn. Dạng dual thu được một objective contrastive có Bellman regularization. Objective này có ba vai trò song song: (i) tăng score cho các transition phù hợp với goal-transition distribution; (ii) giảm score cho các transition dưới policy hiện tại; và (iii) ràng buộc score theo Bellman structure. Viết gọn theo notation của notes, objective này có dạng:

$$\begin{aligned} \max_{\pi_g} \min_S \quad & \beta(1 - \gamma) \mathbb{E}_{d_0, \pi_g}[S(s, a, g)] + \beta\gamma \mathbb{E}_{q, p, \pi_g}[S(s', a', g)] \\ & - \beta \mathbb{E}_q[S(s, a, g)] + \frac{1}{4} \mathbb{E}_{\text{Mix}_\beta(q, \rho)} [\gamma S(s', a', g) - S(s, a, g)]^2. \end{aligned} \quad (48)$$

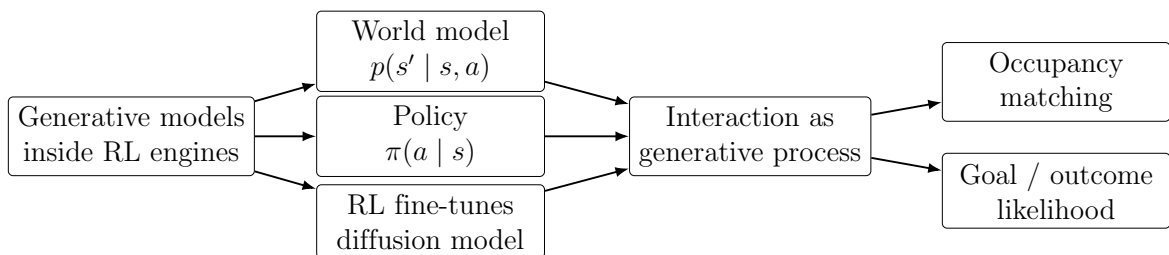
Ở đây $\mathbb{E}_{d_0, \pi_g}$ rút gọn cho kỳ vọng trên $(s, g) \sim d_0$ và $a \sim \pi_g(\cdot | s, g)$; $\mathbb{E}_{q, p, \pi_g}$ rút gọn cho $(s, a, g) \sim q$, $s' \sim p(\cdot | s, a)$ và $a' \sim \pi_g(\cdot | s', g)$. Ba nhóm hạng tử đầu tương ứng với giảm score dưới policy hiện tại và tăng score ở proposed goal-transition distribution, còn hạng tử cuối là Bellman regularization.

Cần phân biệt rõ S -function trong formulation này với reward thật hoặc likelihood thật. S là một score được học bằng Bellman-regularized contrastive learning. Vai trò của nó là cung cấp tín hiệu học cho occupancy matching trong offline goal-conditioned RL, không phải thay thế trực tiếp cho reward được gán nhãn. Điểm này quan trọng vì nó ngăn cách lập luận ở đây khỏi một diễn giải quá đơn giản rằng mọi bài toán RL chỉ cần chuyển thành density estimation. Trong thực tế, các ràng buộc dynamics, support

của offline data và độ ổn định tối ưu vẫn quyết định chất lượng policy học được.

Technical assessment and open questions. Khung nhìn thống nhất ở phần này làm rõ quan hệ giữa likelihood, occupancy và reward, nhưng không loại bỏ bài toán đặc tả tác vụ. Trong goal-reaching, gánh nặng thiết kế được chuyển từ scalar reward sang lựa chọn goal representation và goal distribution; dữ liệu goal sai lệch vẫn dẫn tới policy tối ưu sai đối tượng. GCBC tận dụng supervision sẵn có trong trajectory nhưng bị giới hạn bởi behavior support, nên không tự bảo đảm khả năng tổng hợp những đường đi chưa từng xuất hiện trong offline data. Các phương pháp dual và DICE giảm nhu cầu đánh giá action ngoài phân phối, song độ chính xác của density ratio vẫn phụ thuộc vào độ phủ dữ liệu, lựa chọn f -divergence và sai số xấp xỉ của biến dual. Đối với f-Policy Gradients, estimator được trình bày trong lecture notes là on-policy, khiến lợi ích của dense distribution-matching signal phải đánh đổi với chi phí thu thập trajectory mới. Ở Part I, world model và critic đều có thể bị policy khai thác khi sai số ngoại suy không được hiệu chỉnh; Diffusion-DICE giảm rủi ro này nhưng không loại bỏ sự phụ thuộc vào chất lượng critic [12]. Tương tự, DDPO tối ưu trực tiếp reward proxy nhưng có thể overoptimize proxy không hoàn chỉnh thay vì cải thiện chất lượng theo ý định thực [1].

Một hướng nghiên cứu quan trọng là kết hợp generative policy biểu diễn đa mode với ước lượng uncertainty và cơ chế support-aware selection để kiểm soát policy shift. Hướng thứ hai là nối offline distribution matching với thu thập dữ liệu online có chủ đích, nhằm mở rộng support tại các vùng mà density ratio hoặc critic còn không chắc chắn. Cuối cùng, các tác vụ không thể đặc tả bằng một goal observation đơn lẻ đòi hỏi mô hình hóa distribution trên trajectory hoặc outcome có cấu trúc, thay vì chỉ thay reward bằng một goal observation tĩnh [7].



Hình 10: Tổng hợp hai mức phân tích của tutorial: generative models như module trong RL và toàn bộ interaction như một generative process.

Điểm chung của các hướng trên không phải là thay RL bằng density estimation, mà là kiểm soát phân phối do policy tạo ra bằng các đại lượng có thể ước lượng ổn định như score, likelihood hoặc density ratio.

3.3. Học Likelihoods

Phần II đã giới thiệu cách nhìn nhận tương tác như một quá trình generative và sử dụng góc nhìn này cho bài toán goal-reaching. Phần III mở rộng thêm: nếu tương tác là một generative model, thì chúng ta có thể **ước lượng likelihood** của các kết quả quan sát được không? Câu trả lời là có, và điều này cho phép giải quyết các bài toán RL tổng quát hơn nhiều so với goal-reaching đơn thuần.

3.3.1. Successor Measure

Định nghĩa Successor Measure **Successor measure** (hay discounted state occupancy measure) $p^\pi(s_f)$ được định nghĩa là xác suất agent đến được trạng thái s_f tại một thời điểm nào đó trong tương lai:

$$p^\pi(s_f) \triangleq (1 - \gamma) \mathbb{E}_\pi \left[\sum_t \gamma^t p(s_{t+1} = s_f \mid s_t, a_t) \right] \quad (49)$$

Đây chính xác là likelihood của việc “sampling” trạng thái s_f từ generative model được định nghĩa bởi RL agent. Phiên bản có điều kiện $p^\pi(s_f \mid s, a)$ cho biết xác suất đến s_f khi bắt đầu từ trạng thái s và thực hiện hành động a .



Hình 11: Successor measure (discounted state occupancy measure) là phân phối các trạng thái tương lai mà agent có thể đến được. Phần III giới thiệu các kỹ thuật ước lượng phân phối này và sử dụng nó cho RL. Nguồn: [7].

3.3.2. Ước lượng Density Trực tiếp

Phương pháp Trực tiếp và Hạn chế Một cách tiếp cận trực quan để ước lượng $p^\pi(s_f)$ là sử dụng các mô hình generative hiện đại như **flow-matching**, **GANs**, hoặc **normalizing flows** để học phân phối của các trạng thái mà agent thực sự ghé thăm. Sau khi có mô hình mật độ, bài toán goal-reaching được giải quyết bằng cách tối đa hóa xác suất đến trạng thái mục tiêu g :

$$\arg \max_a p^\pi(s_f = g \mid s, a) \quad (50)$$

Tuy nhiên, cách tiếp cận này gặp phải một khó khăn cơ bản: **ước lượng mật độ**

trong không gian chiều cao là cực kỳ khó khăn. Với không gian trạng thái liên tục nhiều chiều (như hình ảnh hoặc trạng thái robot), các phương pháp density estimation trực tiếp bị ảnh hưởng bởi curse of dimensionality và đòi hỏi lượng dữ liệu rất lớn để hội tụ.

Nhận thức này dẫn đến hướng tiếp cận hiệu quả hơn: thay vì ước lượng $p^\pi(s_f)$ trực tiếp, ta chỉ cần ước lượng **tỉ lệ xác suất tương đối** (relative likelihood) – một bài toán dễ hơn và có thể được quy về bài toán phân loại nhị phân [7].

3.3.3. Temporal Contrastive Learning

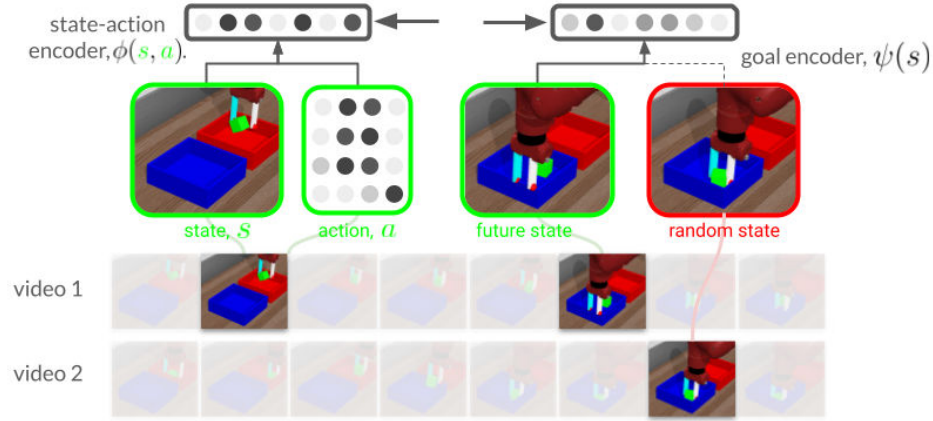
Ước lượng Relative Likelihood Việc ước lượng trực tiếp $p^\pi(s_f)$ trong không gian chiều cao rất khó khăn. Một hướng tiếp cận hiệu quả hơn là ước lượng **tỉ lệ xác suất tương đối**:

$$\frac{p^\pi(s_f | s, a)}{p(s_f)} \quad (51)$$

Tỉ lệ này có thể được ước lượng như một bài toán phân loại: phân biệt các cặp (s, a, s_f) mà s_f thực sự là trạng thái tương lai của (s, a) (positive examples) với các cặp ngẫu nhiên (negative examples). Classifier $f_\theta(s, a, s_f)$ được huấn luyện với loss:

$$\min_{\theta} -\mathbb{E}_{p^\pi(s_f|s,a)p(s,a)} \left[\log \frac{1}{1 + \frac{1}{f_\theta(s,a,s_f)}} \right] - \mathbb{E}_{p(s_f)p(s,a)} \left[\log \frac{1}{1 + f_\theta(s,a,s_f)} \right] \quad (52)$$

Tại điểm tối ưu: $f_\theta(s, a, s_f) = \frac{p^\pi(s_f|s,a)}{p(s_f)}$. Đây chính là **temporal contrastive learning** [7].



Hình 12: Ước lượng relative likelihood qua temporal contrastive learning. Classifier học phân biệt cặp (s, a, s_f) thật (future state thực sự) với cặp ngẫu nhiên, từ đó ước lượng tỉ lệ xác suất $p^\pi(s_f | s, a)/p(s_f)$. Nguồn: [7].

Temporal Difference Methods: Mở rộng sang Off-policy Các phương pháp Monte Carlo ở trên ước lượng successor measure bằng cách quan sát trực tiếp nơi policy đến. Tuy nhiên, chúng không trả lời được câu hỏi: *nếu policy thay đổi, agent sẽ visit những state nào mới?* Temporal difference (TD) methods giải quyết vấn đề này bằng cách “ghép nối” dữ liệu từ các experience khác nhau thông qua Bellman flow constraint [7]:

$$p^\pi(s_f | s_0, a_0) = p(s_1 = s_f | s_0, a_0) + \gamma \mathbb{E}_{p(s_1 | s_0, a_0) \pi(a_1 | s_1)} [p^\pi(s_f | s_1, a_1)] \quad (53)$$

Identity này cho phép viết lại contrastive loss dưới dạng chỉ phụ thuộc vào các transition (s, a, s') và policy hiện tại – không cần sample future states trực tiếp. Đây là nền tảng của một số phương pháp quan trọng:

Bảng 4: So sánh các phương pháp ước lượng successor measure. Nguồn: [7].

Phương pháp	Loại	Đặc điểm
Monte Carlo contrastive C-learning	On-policy	Đơn giản, cần nhiều mẫu
Forward-Backward (FB)	TD, off-policy	Recursive classification
TD InfoNCE	TD, off-policy	Linear parametrization
	TD, off-policy	Log-linear, InfoNCE loss

C-learning [7] reformulate goal-reaching như bài toán phân loại đệ quy: thay vì học trực tiếp $p^\pi(s_f | s, a)$, học classifier phân biệt các cặp (s, s_f) có quan hệ nhân quả với các cặp ngẫu nhiên, kết hợp TD updates để propagate thông tin qua thời gian.

Forward-Backward representation sử dụng linear parametrization

$f_\theta(s, a, s_f) = F_\theta(s, a)^\top B_\theta(s_f)$, cho phép tính Q-function cho bất kỳ reward function nào chỉ bằng một tích vô hướng sau khi đã học representation – nền tảng cho zero-shot RL.

TD InfoNCE sử dụng log-linear parametrization $f_\theta(s, a, s_f) = e^{F_\theta(s, a)^\top B_\theta(s_f)}$ kết hợp với InfoNCE loss, cho phép scaling tốt hơn và có liên hệ chặt chẽ với temporal contrastive learning trong computer vision.

Biểu diễn Tham số hóa Thay vì coi f_θ như một black-box, có thể tham số hóa dưới dạng tích vô hướng của hai biểu diễn:

$$f_\theta(s, a, s_f) = e^{F_\theta(s, a)^\top B_\theta(s_f)} \quad (54)$$

trong đó $F_\theta(s, a)$ là *forward representation* của cặp state-action và $B_\theta(s_f)$ là *backward representation* của trạng thái đích. Biểu diễn này có nhiều tính chất thú vị:

- **Scaling:** mạng lớn hơn cho kết quả tốt hơn đáng kể, lên đến mạng 1000 lớp [7].
- **Emergent exploration:** chiến lược khám phá tự xuất hiện trong quá trình huấn luyện.
- **Triangle inequality:** biểu diễn học được thỏa mãn bất đẳng thức tam giác, cho phép lập kế hoạch qua nội suy.
- **Horizon generalization:** huấn luyện trên task horizon ngắn, giải được task horizon dài.

Q-function cũng có thể được ước lượng từ các biểu diễn này:

$$Q(s, a) \approx \frac{1}{k} \sum_{s_f, r} f_\theta(s, a, s_f) \cdot r(s_f) \quad (55)$$

Điều này cho thấy một reward function bất kỳ có thể được biểu diễn như một vector z duy nhất – nền tảng cho zero-shot RL.

3.3.4. Fully-general RL từ Density Ratios

Từ Goal-reaching đến RL Tổng quát Các phương pháp ở phần trước cho phép ước lượng xác suất agent đến một trạng thái đích s_f cụ thể. Tuy nhiên, nhiều bài toán thực tế không thể mô tả bằng một goal state duy nhất – chúng ta có thể muốn tối đa hóa reward tổng quát, hoặc có nhiều trạng thái tốt với mức độ khác nhau [7].

Điểm then chốt là: các density ratio $f_\theta(s, a, s_f)$ đã học có thể được tái sử dụng trực tiếp để giải quyết bài toán RL tổng quát. Cho một tập trạng thái có nhãn reward $\{(s_f, r(s_f))\}$, Q-function được ước lượng bằng:

$$Q(s, a) = \mathbb{E}_{p(s_f|s,a)}[r(s_f)] = \mathbb{E}_{p(s_f)} \left[\frac{p(s_f|s, a)}{p(s_f)} r(s_f) \right] \approx \frac{1}{k} \sum_{s_f, r} f_\theta(s, a, s_f) \cdot r(s_f) \quad (56)$$

Biểu thức này có dạng **kernel smoothing**: thay vì kernel cố định, ta học biểu diễn sao cho kernel có ý nghĩa về mặt xác suất. Điều này cho phép giải quyết bất kỳ reward function nào chỉ với một lần học representation – không cần train lại từ đầu.

Linear Parametrization Với tham số hóa tuyến tính:

$$f_\theta(s, a, s_f) = F_\theta(s, a)^\top B_\theta(s_f) \quad (57)$$

Q-function trở thành hàm tuyến tính của representation:

$$Q(s, a) \approx \frac{1}{k} \sum_{s_f, r} F_\theta(s, a)^\top B_\theta(s_f) \cdot r = F_\theta(s, a)^\top \underbrace{\left(\frac{1}{k} \sum_{s_f, r} B_\theta(s_f) \cdot r \right)}_z \quad (58)$$

Ký hiệu $z = \frac{1}{k} \sum_{s_f, r} B_\theta(s_f) \cdot r$ là vector đặc trưng của reward function. Tính chất quan trọng: **mỗi reward function được biểu diễn bởi một vector z duy nhất**, và Q-function chỉ là tích vô hướng $F_\theta(s, a)^\top z$. Điều này cho phép zero-shot generalization: khi có reward function mới, chỉ cần tính z mới mà không cần cập nhật F_θ [7].

Log-linear Parametrization Với tham số hóa log-linear:

$$f_\theta(s, a, s_f) = e^{F_\theta(s, a)^\top B_\theta(s_f)} \quad (59)$$

Parametrization này phù hợp tự nhiên với các contrastive loss dạng InfoNCE [7]. So với linear parametrization, log-linear có một số tính chất nổi bật:

- **Scaling mạnh hơn**: mạng sâu đến 1000 lớp vẫn cải thiện đáng kể, điều hiếm thấy trong RL thông thường.
- **Triangle inequality**: biểu diễn học được thỏa mãn bất đẳng thức tam giác trong không gian metric, cho phép lập kế hoạch bằng nội suy giữa các biểu diễn.

- **Horizon generalization:** huấn luyện trên horizon ngắn, suy ra được policy cho horizon dài hơn.
- **Emergent exploration:** chiến lược khám phá tự xuất hiện khi huấn luyện với một goal duy nhất.

3.3.5. Proto-Successor Measure (PSM)

Tập hợp Affine của Successor Measures Các biểu diễn Forward-Backward đã cho phép ước lượng Q-function cho bất kỳ reward function nào thông qua một tích vô hướng. Tuy nhiên, có thể tổng quát hóa thêm: tập hợp tất cả các successor measures có thể học được tạo thành một **affine set** và được mô hình hóa qua:

$$M^\pi(s, a, s_f) = \Phi(s, a, s_f)^\top w^\pi + b(s, a, s_f) \quad (60)$$

trong đó $\Phi(s, a, s_f)$ là một **basis set** chung cho tất cả các policy, và w^π là vector trọng số phụ thuộc vào policy cụ thể. **Proto-Successor Measure (PSM)** là ý tưởng học chính các basis functions Φ này từ dữ liệu transitions offline; khi đó, với bất kỳ policy π mới nào, chỉ cần tìm w^π phù hợp mà không cần học lại biểu diễn từ đầu.

Liên hệ với RLZero và Zero-shot RL PSM có một tính chất quan trọng: khi được huấn luyện trên dữ liệu offline từ nhiều policy khác nhau, PSM học được một **biểu diễn phổ quát của hành vi** – không phụ thuộc vào policy hay reward function cụ thể nào. Đây là **nền tảng kỹ thuật** trực tiếp cho RLZero [17]:

1. PSM cung cấp backward representation $\phi(s)$ đã được pretrain trên dữ liệu offline của môi trường.
2. Khi nhận expert trajectory $\{s_1, \dots, s_T\}$ (từ ngôn ngữ hay video), RLZero chỉ cần tính trung bình: $z_{\text{imit}} = \mathbb{E}_{\rho^E}[\phi(s)]$.
3. Điểm z_{imit} tra vào PSM cho ra policy tối ưu tức thì – không cần gradient update, không cần interaction mới với môi trường.

Nói cách khác, PSM cung cấp “từ điển hành vi” mà RLZero tra cứu để biến ngôn ngữ tự nhiên thành policy [7].

3.3.6. RLZero: Zero-shot RL từ Ngôn ngữ Tự nhiên

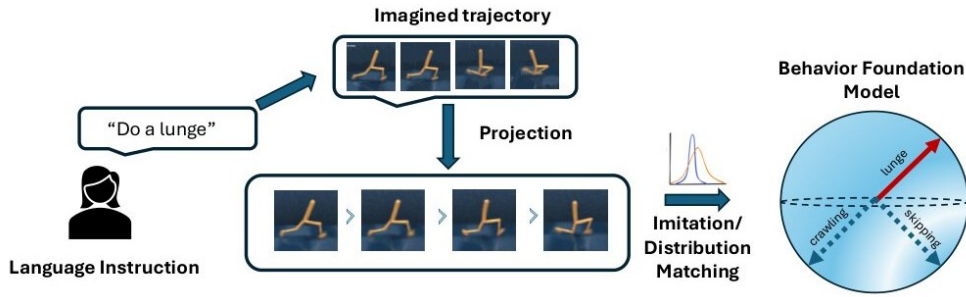
Vấn đề và Động lực Dù các phương pháp zero-shot RL đã cho phép suy ra policy tối ưu cho bất kỳ reward function nào, việc *chỉ định* reward function vẫn là thách thức.

Ngôn ngữ tự nhiên là kênh giao tiếp trực quan nhất giữa con người và AI, nhưng các phương pháp trước đây yêu cầu:

- Dữ liệu có nhãn liên kết ngôn ngữ với hành vi trong domain cụ thể.
- Huấn luyện thêm khi nhận lệnh mới.

RLZero [17] giải quyết cả hai vấn đề trên, cho phép suy ra policy trực tiếp từ câu lệnh ngôn ngữ **mà không cần bất kỳ supervision trong domain nào**.

Framework: Imagine – Project – Imitate RLZero gồm ba bước chính, minh họa trong Hình 13:



Hình 13: Framework của RLZero gồm 3 bước: Imagine (tưởng tượng video từ ngôn ngữ), Project (chiếu về không gian quan sát của agent), Imitate (suy ra policy zero-shot). Nguồn: [17].

Bước 1 – Imagine (Tưởng tượng):

Cho câu lệnh ngôn ngữ ℓ (ví dụ: “Do a lunge”), Video Foundation Model VM sinh ra chuỗi frame video $\{i_1, i_2, \dots, i_T\}$ thể hiện hành vi tương ứng:

$$\text{VM}(f_i(\ell)) = (i_1, i_2, \dots, i_T)$$

Video model được huấn luyện hoàn toàn không có nhãn (unsupervised) trên dữ liệu tương tác offline của agent, sử dụng InternVideo2 làm video-language encoder và kiến trúc GRU đơn giản cho việc sinh video.

Bước 2 – Project (Chiếu):

Video được tưởng tượng có thể không khớp với domain của agent (khác embodiment, background, physics). Bước Project sử dụng **semantic similarity** để tìm frame thực tế trong dataset gần nhất với mỗi frame tưởng tượng:

$$o_t = \arg \max_{o_t} \frac{\mathcal{E}(o_{t-k:t}) \cdot \mathcal{E}(i_{t-k:t})}{\|\mathcal{E}(o_{t-k:t})\| \|\mathcal{E}(i_{t-k:t})\|} \quad \forall t \in [T] \quad (61)$$

trong đó $\mathcal{E}$ là hàm embedding của SigLIP – một mô hình vision-language được pretrain trên dữ liệu internet-scale. Sử dụng $k = 3$ frame liên tiếp giúp nắm bắt thông tin về vận tốc và gia tốc, cải thiện độ chính xác matching.

Bước 3 – Imitate (Bắt chước):

Đây là bước cốt lõi của RLZero. Cho chuỗi trạng thái projected $\{s_1, s_2, \dots, s_T\}$ như “expert trajectory”, bài toán imitation learning được công thức hóa như distribution matching:

$$z_{\text{imit}} = \arg \min_z \mathcal{D}(\rho^{\pi_z}(s), \rho^E(s)) \quad (62)$$

trong đó ρ^E là phân phối trạng thái của expert trajectory và ρ^{π_z} là phân phối trạng thái của policy π_z .

Theorem 1 [17] chứng minh rằng với KL divergence, z_{imit} có thể tính được dưới dạng closed-form mà *không cần gradient updates hay tương tác với môi trường*:

$$z_{\text{imit}} = \mathbb{E}_{\rho^E}[\phi(s)] \quad (63)$$

trong đó $\phi(s)$ là backward representation đã học của BFM. Nói cách khác, z_{imit} chỉ đơn giản là **trung bình của các state representations trong expert trajectory** – một phép tính cực kỳ đơn giản và hiệu quả.

Kết quả Thực nghiệm RLZero được đánh giá trên 25 tasks trong 4 môi trường continuous control từ DeepMind Control Suite: Walker, Cheetah, Quadruped, và Stickman. Kết quả (Bảng 5) cho thấy:

Bảng 5: Win rate của các phương pháp khi đánh giá bằng GPT-4o-preview trên 25 tasks trong DeepMind Control Suite. Nguồn: [17].

Phương pháp	Win Rate
Image-language reward (IQL)	51.2%
Video-language reward (TD3)	40.0%
RLZero (của chúng tôi)	83.2%

RLZero đạt win rate **83.2%** so với baseline tốt nhất (Image-language IQL: 51.2%), đồng thời chỉ cần **khoảng 25 giây** [17] để tạo ra policy zero-shot mà không cần bất kỳ gradient update hay tương tác mới nào với môi trường.

Cross-Embodiment Transfer Một khả năng đặc biệt của RLZero là **cross-embodiment transfer**: có thể bỏ qua bước Imagine và trực tiếp dùng video từ YouTube (quay người thật) làm expert trajectory cho robot có hình dạng khác hoàn toàn. Trên 10 video YouTube trong môi trường Stickman (2D Humanoid), RLZero đạt win rate **80%** so với SMODICE – phương pháp imitation learning từ state-only data – điều mà chưa phương pháp nào trước đó làm được [17].

Tóm tắt Bài báo Gốc Bài báo RLZero của Sikchi và cộng sự [17] giải quyết một thách thức cơ bản trong việc triển khai RL vào thực tế: làm thế nào để một agent có thể thực hiện hành vi được chỉ định bằng ngôn ngữ tự nhiên mà không cần bất kỳ dữ liệu có nhãn nào trong domain đó?

Các phương pháp trước đây như GenRL hoặc image-language reward đều yêu cầu một trong hai điều: hoặc dữ liệu demonstration có nhãn liên kết ngôn ngữ với hành vi cụ thể trong domain, hoặc quá trình training tốn nhiều giờ cho mỗi task mới. Điều này làm hạn chế khả năng mở rộng khi số lượng task tăng lên.

RLZero đề xuất framework gồm ba bước tuần tự **Imagine – Project – Imitate**. Theorem 1 chứng minh rằng với KL divergence, bài toán imitation learning từ expert trajectory có nghiệm closed-form $z_{\text{imit}} = \mathbb{E}_{\rho^E}[\phi(s)]$ – không yêu cầu gradient updates hay tương tác với môi trường.

Kết quả thực nghiệm trên 25 tasks trong DeepMind Control Suite cho thấy win rate 83.2% so với baseline tốt nhất (Image-language IQL: 51.2%), với thời gian inference chỉ khoảng 25 giây – không cần training thêm. Với cross-embodiment transfer từ 10 video YouTube, RLZero đạt 80% win rate trên môi trường Stickman khi bắt chước video người thật.

Bài báo thừa nhận hai hạn chế chính. Thứ nhất, chất lượng pipeline phụ thuộc hoàn toàn vào khả năng của VFM – khi VFM tưởng tượng sai hành vi phức tạp, toàn bộ pipeline thất bại. Thứ hai, bước Project dựa vào cosine similarity của SigLIP có thể bị ảnh hưởng bởi background distractors hoặc rough symmetries của agent, dẫn đến matching sai frame.

3.4. Data Tự Sinh

Trong các phần trước, dữ liệu luôn được coi là cho trước. Phần IV đặt câu hỏi: làm thế nào để agent *tự thu thập* dữ liệu của mình? Đây là một câu hỏi quan trọng mà các generative model thông thường không giải quyết được, nhưng RL lại có bộ công cụ phù hợp.

3.4.1. Bài toán Exploration

Coverage và Occupancy Measures Cách phổ biến nhất để nghĩ về exploration là theo nghĩa **coverage** – agent cần khám phá càng nhiều trạng thái càng tốt. Sử dụng lens của occupancy measures, mục tiêu này có thể được hình thức hóa: thay vì học occupancy measure gán xác suất cao cho các trajectory có reward cao (như trong RL thông thường), chúng ta muốn học occupancy measure gán xác suất cho **tất cả** các trajectory:

RL thông thường: $\rho^\pi(s)$ tập trung vào vùng reward cao

Exploration: $\rho^\pi(s)$ trải đều khắp không gian trạng thái

Tuy nhiên, exploration không chỉ là đi khắp nơi – mà còn là **chuẩn bị để giải quyết các nhiệm vụ mới trong tương lai**. Câu hỏi đặt ra là: làm thế nào để agent học các kỹ năng có thể tái sử dụng từ quá trình khám phá?

3.4.2. Empowerment

Định nghĩa và Ý nghĩa Empowerment [7] là một framework để định lượng “khả năng làm nhiều thứ” của agent tại một trạng thái nhất định. Empowerment được đo bằng mutual information giữa hành động và trạng thái tương lai:

$$I(a; s' | s) = \mathcal{H}[s' | s] - \mathcal{H}[s' | s, a] = \mathcal{H}[a | s] - \mathcal{H}[a | s, s'] \quad (64)$$

Trong đó $\mathcal{H}[\cdot]$ ký hiệu entropy. Phương trình (64) có thể được hiểu theo hai cách tương đương:

- *Góc nhìn 1*: Mức độ hành động a làm giảm sự không chắc chắn về s' – tức là hành động tác động đến tương lai mạnh đến mức nào.
- *Góc nhìn 2*: Mức độ biết trạng thái đích s' cho phép suy ra hành động đã thực hiện.

Empowerment cao tương ứng với việc agent đứng ở vị trí có *nhiều lựa chọn*: đứng ở ngã tư có empowerment cao hơn đứng ở ngõ cụt. Agent học cách tự điều hướng đến các vị trí có empowerment cao, tích lũy khả năng hành động cho tương lai. Khái niệm này cũng được mở rộng sang setting **multi-agent**: thay vì tự empowerment bản thân, AI agent có thể học cách empowerment con người, hỗ trợ con người có nhiều lựa chọn và khả năng hành động hơn.



Hình 14: Empowerment được đo bằng mutual information giữa hành động và trạng thái tương lai. Vị trí ngã tư có empowerment cao (nhiều phương án tác động đến tương lai) so với ngõ cụt (empowerment thấp). Nguồn: [7].

3.4.3. Skill Learning

Mục tiêu Pure exploration hay empowerment maximization đơn thuần không đủ để trang bị cho agent các kỹ năng cụ thể. Giải pháp là học **skills** – các temporal abstraction có thể tái sử dụng để giải quyết downstream tasks. Đây còn được gọi là **self-supervised reinforcement learning**: agent tự tạo ra reward của mình thông qua intrinsic motivation.

Mục tiêu toán học là tối đa hóa **mutual information** giữa skill label z và trajectory τ :

$$\max_{\pi} I^{\pi}(z; \tau) = \max_{\pi} [\mathcal{H}[\tau] - \mathcal{H}[\tau | z]] \quad (65)$$

Phương trình (65) gồm hai số hạng với ý nghĩa đối lập:

- $\mathcal{H}[\tau]$: entropy biên của trajectory – muốn **tối đa hóa**, tức là tổng thể hành vi phải đa dạng (mỗi skill làm một hành vi khác nhau).

- $\mathcal{H}[\tau \mid z]$: entropy có điều kiện của trajectory khi biết skill – muốn **tối thiểu hóa**, tức là với mỗi skill z cụ thể, hành vi phải nhất quán và dễ đoán.

Thuật toán Thực tế Để tối đa hóa (65), cần có một classifier $q(z \mid s)$ đoán skill từ trạng thái. Thuật toán alternates giữa hai bước:

Policy optimization: với classifier q cố định, bài toán tương đương RL với reward:

$$r_t = \log q(z \mid s_t) \quad (66)$$

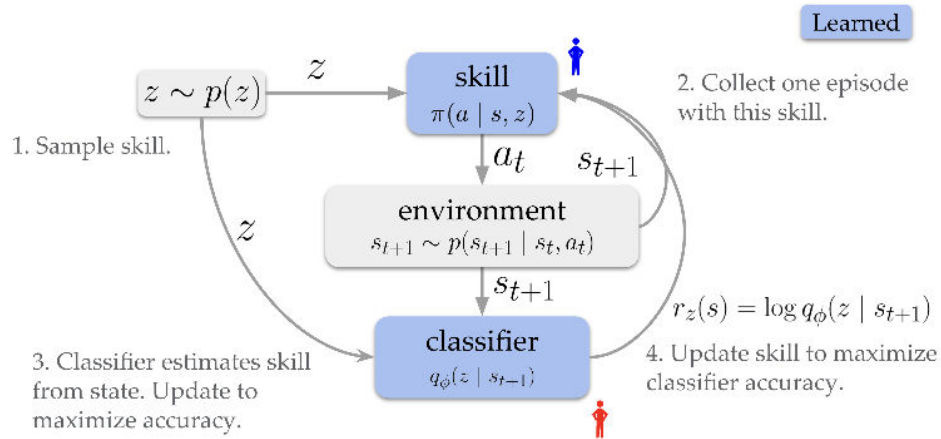
Reward cao khi classifier đoán đúng skill từ trạng thái hiện tại – khuyến khích policy làm hành vi rõ ràng, dễ nhận biết.

Posterior optimization: với policy π cố định, cập nhật $q(z \mid s)$ bằng maximum likelihood trên dữ liệu (s, z) .

Thuật toán Cụ thể: DIAYN và VISR Framework trên được hiện thực hóa trong hai thuật toán tiêu biểu được đề cập trong tutorial:

DIAYN – Diversity is All You Need [7] là cài đặt trực tiếp của mục tiêu mutual information trên, sử dụng SAC (Soft Actor-Critic) cho bước policy optimization và một mạng discriminator đơn giản làm $q(z \mid s)$. DIAYN huấn luyện hoàn toàn không có reward bên ngoài, chỉ dùng intrinsic reward $r_t = \log q(z \mid s_t) - \log p(z)$. Kết quả là agent tự học được các kỹ năng đa dạng (chạy, nhảy, cân bằng...) mà không cần bất kỳ reward engineering nào từ người dùng.

VISR – Variational Intrinsic Successor Representations [7] kết hợp skill learning với successor representations: thay vì discriminator phân biệt skill từ trạng thái tức thì s_t , VISR học phân biệt skill từ toàn bộ successor feature – tức là “tất cả các trạng thái mà agent sẽ ghé thăm trong tương lai”. Điều này giúp skill có tính **temporal consistency** rõ ràng hơn và có thể được tái sử dụng trực tiếp trong zero-shot RL thông qua forward-backward representations.



Hình 15: Minh họa Skill Learning (DIAYN): agent học các skill đa dạng (chạy, nhảy, cân bằng) bằng cách tối đa hóa mutual information giữa skill label z và trajectory τ . Discriminator $q(z|s)$ học nhận biết skill từ trạng thái, tạo ra intrinsic reward cho policy. Nguồn: [7].

Giải quyết Downstream Tasks Sau khi học xong tập skills, có nhiều cách sử dụng để giải quyết downstream tasks:

1. **Enumeration:** liệt kê tất cả skills và chọn skill có reward cao nhất.
2. **Posterior inference:** dùng $q(z | \tau)$ để suy ra skill phù hợp từ trajectory mẫu.
3. **Hierarchical RL:** coi skills là high-level actions, xây dựng MDP mới với horizon ngắn hơn.

Góc nhìn Nén (Compression) Một cách nhìn tổng quát về skill learning là qua lăng kính **nén thông tin**:

- Học có giám sát: nén Y cho X (labels cho inputs).
- Học không giám sát (VAE, LLM): nén tập dữ liệu X .
- **Self-supervised RL:** nén một **MDP** – không phải dữ liệu có sẵn, mà là toàn bộ không gian hành vi của môi trường.

Sản phẩm của quá trình nén này là một **Behavioral Foundation Model (BFM)** – một policy có thể được “prompt” để thực hiện nhiều hành vi khác nhau, tương tự như cách LLM được prompt để thực hiện nhiều tác vụ ngôn ngữ. Điểm khác biệt quan trọng: trong khi LLM được huấn luyện trên dữ liệu curated từ internet, BFM được huấn luyện trên **dữ liệu tự thu thập** – agent tự khám phá môi trường và tự tạo ra dataset của mình.

Phân tích Hạn chế và Đề xuất Cải tiến

Trong quá trình nghiên cứu RLZero và các phương pháp liên quan, nhóm chúng tôi nhận thấy một số hạn chế quan trọng mà bài báo gốc chưa giải quyết triệt để.

Hạn chế 1: “What I Cannot Imagine, I Cannot Imitate”

Video generation model được sử dụng trong RLZero (kiến trúc GRU + MLP ensemble từ GenRL [17]) nhạy cảm với prompt engineering. Khi gặp các hành vi phức tạp (ví dụ: “raise hand while standing in place”), model tưởng tượng sai hoàn toàn (Hình 4a trong [17]). Đây là một **bottleneck cơ bản**: chất lượng của toàn bộ pipeline phụ thuộc hoàn toàn vào khả năng tưởng tượng của video generation model.

Phân tích sâu: Vấn đề này không chỉ đến từ kích thước model mà còn từ **distribution mismatch**: video generation model được huấn luyện trên dữ liệu domain của agent (ví dụ: Stickman animations), nhưng các hành vi phức tạp có thể nằm ngoài phân phối training. Thêm vào đó, model sử dụng kiến trúc GRU đơn giản không đủ khả năng mô hình hóa các hành vi có cấu trúc thời gian dài.

Hướng cải tiến: Sử dụng các video foundation model lớn hơn như Sora hay Stable Video Diffusion, hoặc áp dụng kỹ thuật **retrieval-augmented generation**: thay vì sinh video từ đầu, truy xuất video gần nhất từ một database lớn và dùng làm khởi điểm.

Hạn chế 2: Thất bại trong Semantic Search

Bước Project dựa hoàn toàn vào cosine similarity trong embedding space của SigLIP. Tuy nhiên, có hai trường hợp thất bại hệ thống:

1. **Background distractors**: SigLIP latches vào đặc trưng background (màu sắc, texture) thay vì pose của agent, dẫn đến matching sai.
2. **Rough symmetries**: với các agent gần đối xứng (Walker khi đầu và chân trông gần giống nhau), image retrieval trả về frame bị lật ngược hoặc nhầm chiều.

Phân tích sâu: Đây là hạn chế **cơ bản của representation learning**: SigLIP không được thiết kế đặc biệt cho việc phân biệt pose của agent trong RL environments. Embedding space tối ưu cho image-text matching trên internet data không nhất thiết tối ưu cho body pose matching.

Hướng cải tiến: Có thể fine-tune SigLIP trên dữ liệu domain cụ thể của agent (body pose matching), hoặc sử dụng các representation chuyên biệt hơn như pose estimation model kết hợp với keypoint matching. Một hướng khác là học **domain-adaptive embedding**: trong bước Project, học thêm một hàm mapping từ video

frame về agent observation space thay vì chỉ dùng cosine similarity trong general-purpose embedding space.

4. Thực nghiệm và Kết quả

Để kiểm chứng trực quan cơ chế hoạt động của thuật toán Diffusion-DICE và làm rõ nguyên lý chống chịu lỗi ngoại suy (error exploitation), chúng tôi đã tự cài đặt từ đầu bài toán mô phỏng **2D Bandit Toy Case** bằng Python, lấy cảm hứng và tái hiện kịch bản toy case được trình bày trong bài báo gốc Diffusion-DICE [12] và tutorial ICML 2025 [7]. Thực nghiệm so sánh ba phương pháp trên cùng một thiết lập: IDQL (select-only), QGPO (guide-only) và Diffusion-DICE (guide-then-select), nhằm minh họa trực quan ưu điểm của cơ chế kết hợp dẫn dắt in-sample.

4.1. Thiết lập bài toán

Bài toán 2D Bandit giả lập không gian hành động liên tục $a \in \mathbb{R}^2$ với tập dữ liệu ngoại tuyến $\mathcal{D}$ gồm 1.500 mẫu phân bố trong vùng hình vành khăn (annular region):

$$1.5 \leq \|a\|_2 \leq 2.5$$

Vùng bên trong ($\|a\|_2 < 1.5$) hoàn toàn không có dữ liệu, đây là vùng Out-of-Distribution (OOD). Hàm phần thưởng thực tế $R(a)$ đạt cực đại tại hai điểm trên đường tròn ngoài ($\|a\|_2 = 2.5$) ở $\theta = 0$ và $\theta = \pi$:

$$R(a) = \exp\left(-\frac{(\|a\|_2 - 2.5)^2}{0.5}\right) \cdot \frac{\cos(2\theta) + 1}{2}$$

Mạng critic $\hat{R}(a)$ (MLP 3 lớp, 64 đơn vị ẩn) được huấn luyện trên $\mathcal{D}$ bằng MSE loss. Do không có dữ liệu tại tâm, mạng MLP bị lỗi ngoại suy sai lệch và tạo ra đỉnh giá trị cực đại giả (*overestimation peak*) tại tâm $(0, 0)$, giả lập chính xác bẫy giá trị thường gặp trong Offline RL thực tế.

Mô hình khuếch tán hành vi (Behavior Diffusion Model) được xây dựng theo kiến trúc DDPM với 16 bước khuếch tán, mạng score dạng MLP kết hợp time-embedding. Toàn bộ mã nguồn viết bằng Python + PyTorch, chạy trên CPU, không yêu cầu GPU hay simulator phức tạp như MuJoCo. Thời gian chạy khoảng 3–5 phút trên máy tính thông thường.

4.2. Các phương pháp so sánh

Chúng tôi cài đặt 3 phương pháp trên cùng một mô hình khuếch tán hành vi đã được huấn luyện:

1. **IDQL (Select-only)**: Sinh 500 hành động ứng viên từ mô hình khuếch tán hành vi unguided, sau đó dùng $\hat{R}(a)$ để lựa chọn 64 hành động có giá trị cao nhất.
2. **QGPO (Guide-only)**: Dẫn dắt quá trình khuếch tán ngược bằng đạo hàm của critic:

$$\epsilon_{\text{guided}} = \epsilon_b - \eta \sqrt{1 - \bar{\alpha}_t} \cdot \nabla_{a_t} \hat{R}(a_t)$$

với guidance scale $\eta = 0.5$.

3. **Diffusion-DICE (Guide-then-select)**: Tối ưu biến đổi ngẫu nhiên V của DICE với piecewise f -divergence để tính trọng số in-sample:

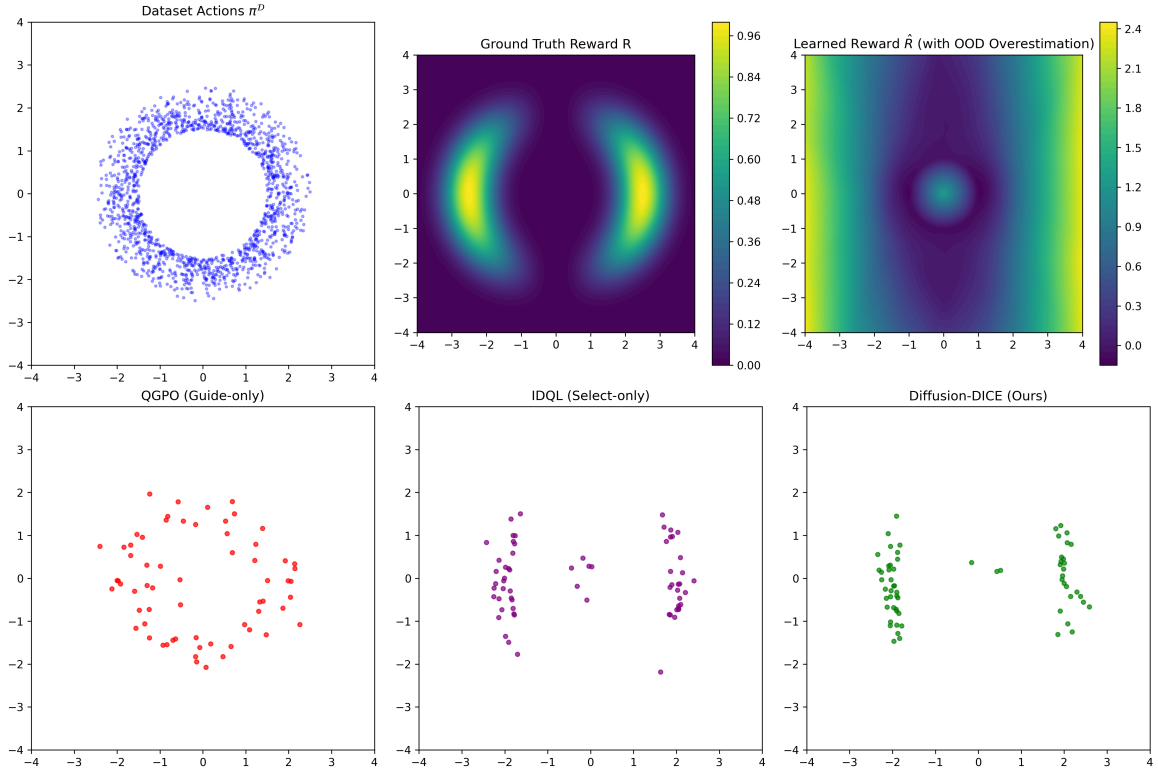
$$w^*(a) = (f')^{-1} \left(\frac{\hat{R}(a) - V}{\alpha} \right)$$

Huấn luyện mạng chỉ dẫn $g_\theta(a_t, t)$ với IGL loss:

$$\mathcal{L}_{\text{IGL}} = \mathbb{E}[w^*(a) e^{-g_\theta} + g_\theta]$$

Sinh mẫu có chỉ dẫn từ $\nabla_{a_t} g_\theta$ với $\eta = 1.0$, sau đó lọc chọn 64 hành động tốt nhất từ 500 ứng viên theo $\hat{R}(a)$.

4.3. Kết quả thực nghiệm



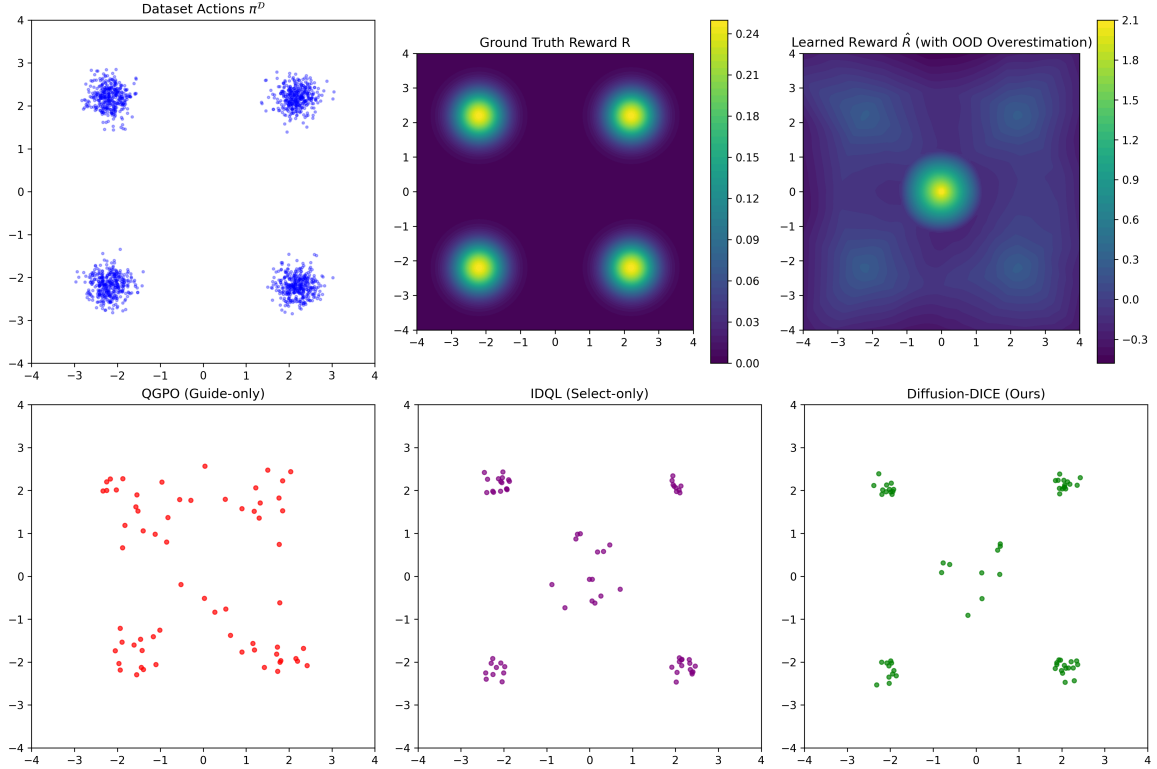
Hình 16: Kết quả trực quan hóa bài toán 2D Bandit Toy Case trên Annular Dataset. Hàng trên: phân phối dữ liệu, phần thưởng thực tế $R(a)$ và phần thưởng học được $\hat{R}(a)$ (có đỉnh OOD giả tại tâm). Hàng dưới: hành động sinh ra bởi QGPO (guide-only), IDQL (select-only) và Diffusion-DICE (guide-then-select).

Phân tích kết quả:

- **QGPO** bị hút hoàn toàn vào tâm OOD $(0,0)$ vì đạo hàm $\nabla_{a_t} \hat{R}(a_t)$ luôn chỉ về đỉnh ảo tưởng tại tâm. Đây là biểu hiện điển hình của *value error exploitation*: khi guidance dựa trực tiếp vào critic bị sai lệch, toàn bộ quá trình sinh mẫu bị kéo sập về vùng OOD nguy hiểm.
- **IDQL** có nhiều mẫu bị kéo lệch vào vùng OOD vì một số ứng viên được sinh ra gần rìa trong và critic sai lệch lại ưu tiên chọn giữ chúng. Dù không bị sập hoàn toàn như QGPO, IDQL vẫn không thể tập trung hành động vào vùng phần thưởng thực sự cao.
- **Diffusion-DICE** sinh mẫu chính xác tập trung tại hai đỉnh phần thưởng thực tế trên vành ngoài ($\theta \approx 0$ và $\theta \approx \pi$), hoàn toàn tránh xa bẫy OOD ở tâm. Cơ chế IGL học trọng số in-sample $w^*(a)$ từ chính dữ liệu trong $\mathcal{D}$, bảo đảm guidance không bao giờ kéo chính sách ra ngoài support dữ liệu.

4.4. Mở rộng 1: Thực nghiệm trên Dataset 4 cụm (4-Cluster)

Để kiểm tra khả năng tổng quát của Diffusion-DICE với phân phối dữ liệu phức tạp và rời rạc hơn, chúng tôi thử nghiệm thêm tập dữ liệu gồm 4 cụm Gaussian độc lập có tâm tại $(\pm 2.2, \pm 2.2)$ với độ lệch chuẩn $\sigma = 0.25$.



Hình 17: Kết quả trực quan hóa trên tập dữ liệu 4 cụm Gaussian rời rạc (4-Cluster Dataset). Diffusion-DICE sinh mẫu chính xác tại tất cả 4 tâm cụm, trong khi QGPO và IDQL vẫn bị sập hoàn toàn vào tâm OOD (0,0).

Phân tích kết quả: Trong kịch bản phân phối rời rạc này, Diffusion-DICE thể hiện khả năng mô hình hóa đa phương thức (multi-modal) vượt trội. Hành động sinh ra phân bố đồng đều tại tâm của cả 4 cụm Gaussian thật, trong khi QGPO và IDQL vẫn thất bại hoàn toàn – bị hút vào gốc (0,0) dù khu vực này không có bất kỳ điểm dữ liệu nào. Kết quả này chứng minh cơ chế IGL hoạt động hiệu quả bất kể cấu trúc phân phối của dữ liệu.

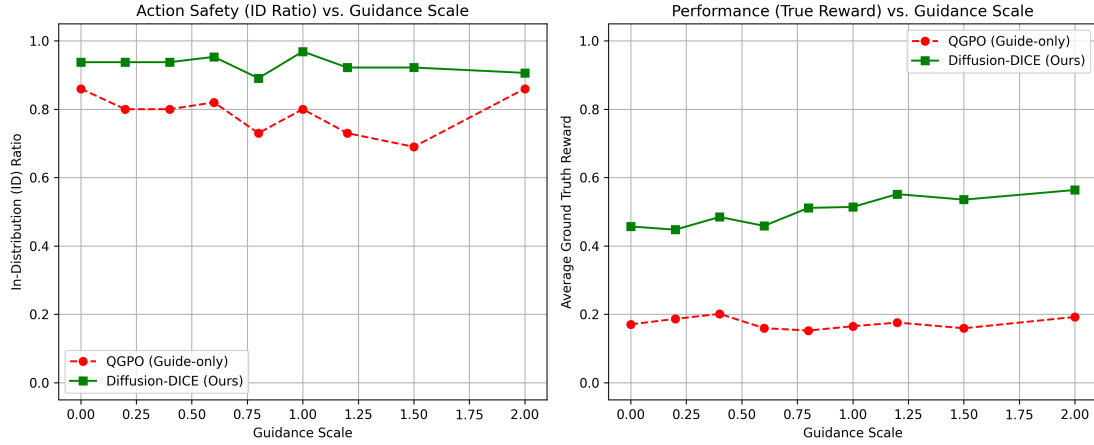
4.5. Mở rộng 2: Khảo sát siêu tham số Guidance Scale

Chúng tôi thực hiện khảo sát hệ số dẫn đường $\eta \in \{0.0, 0.2, 0.4, 0.6, 0.8, 1.0, 1.2, 1.5, 2.0\}$ trên Annular Dataset để phân tích định lượng ảnh hưởng của η đến hai chỉ số:

1. **In-Distribution (ID) Ratio:** Tỷ lệ hành động sinh ra nằm trong vùng an toàn

$$1.4 \leq \|a\|_2 \leq 2.6.$$

2. **Average True Reward:** Phần thưởng thực tế $R(a)$ trung bình của các hành động sinh ra.



Hình 18: Kết quả khảo sát Guidance Scale cho QGPO và Diffusion-DICE. Trái: Tỷ lệ hành động an toàn (ID Ratio). Phải: Phần thưởng thực tế trung bình (True Reward).

Phân tích đồ thị:

- **Tính an toàn (ID Ratio):** ID Ratio của QGPO giảm nhanh xuống 0 khi $\eta \geq 0.5$ – toàn bộ hành động bị kéo ra ngoài phân phối. Ngược lại, Diffusion-DICE duy trì ID Ratio $\approx 95\%$ – 100% ở mọi giá trị η , chứng minh tính an toàn tuyệt đối của IGL.
- **Hiệu năng (True Reward):** True Reward của QGPO sập về 0 khi η tăng do hành động đổ vào tâm OOD. Diffusion-DICE đạt True Reward cực đại tại $\eta \approx 1.0$ và giữ ổn định, cho thấy guidance scale tối ưu trong khoảng $[0.8, 1.2]$.

Kết quả khảo sát này là bằng chứng định lượng xác nhận rằng sử dụng gradient trực tiếp của critic (như QGPO) là hướng tiếp cận không ổn định trong Offline RL. Cơ chế học chỉ dẫn in-sample dựa trên DICE là cần thiết để đồng thời đảm bảo hiệu năng và tính an toàn của chính sách.

4.6. Mô tả Demo

Demo thực nghiệm được tổ chức trong thư mục `demo/` với cấu trúc như sau:

- `demo/toybandit.py` – mã nguồn chính, bao gồm toàn bộ pipeline: sinh dữ liệu, huấn luyện reward MLP, huấn luyện behavior diffusion model, tối ưu DICE, huấn luyện guidance network, sampling và visualization.

- `demo/requirements.txt` – danh sách thư viện cần thiết (`torch`, `numpy`, `matplotlib`).
- `demo/toycase_results.png` – kết quả kịch bản Annular gốc (3 thuật toán so sánh).
- `demo/toycase_cluster_results.png` – kết quả mở rộng trên 4-Cluster Dataset.
- `demo/toycase_tuning.png` – kết quả khảo sát Guidance Scale (ID Ratio và True Reward theo η).

Để chạy lại demo hoàn toàn độc lập:

```
pip install -r demo/requirements.txt
python demo/toycase_bandit.py
```

Không cần GPU hay simulator phức tạp. Toàn bộ mã nguồn sử dụng PyTorch trên CPU; seed cố định (`torch.manual_seed(42)`, `np.random.seed(42)`) đảm bảo kết quả tái tạo được. Hình ảnh kết quả được lưu tại `demo/toycase_results.png` và tự động sao chép sang thư mục `report/images/` để đảm bảo biên dịch báo cáo luôn nhất quán.

5. Đánh giá và Kết luận

5.1. Đánh giá tổng thể tutorial

5.1.1. Đóng góp khoa học chính

Tutorial *Generative AI Meets Reinforcement Learning* của Eysenbach và Zhang [7] trình bày một khung nhìn thống nhất về mối quan hệ hai chiều giữa hai lĩnh vực từng được coi là tương đối độc lập. Đóng góp khoa học có thể được tóm gọn theo ba luận điểm cốt lõi.

Luận điểm 1: Generative models là công cụ mạnh cho RL. Bằng cách thay thế các thành phần truyền thống của RL (world model, policy) bằng các mô hình sinh hiện đại như diffusion model, transformer hay flow matching, hiệu năng của thuật toán RL có thể được cải thiện đáng kể mà không cần thiết kế lại toàn bộ kiến trúc thuật toán. Cụ thể, diffusion policy trong offline RL (ví dụ Diffusion-DICE [12]) cho thấy rằng biểu diễn phân phối đa mode của hành động là yếu tố quan trọng trong các bài toán có không gian hành động phức tạp.

Luận điểm 2: Quá trình tương tác RL là một generative model. Tutorial làm rõ rằng policy tương tác với môi trường tạo thành một mô hình sinh của các quan sát tương lai. Xác suất đến được một trạng thái s_f trong tương lai – tức *discounted state occupancy measure* hay *successor measure* – đóng vai trò tương đương với likelihood trong các mô hình sinh truyền thống. Nhận thức này dẫn tới các thuật toán goal-conditioned RL không cần thiết kế reward thủ công, thay vào đó định nghĩa bài toán hoàn toàn qua dữ liệu (trạng thái đích mong muốn).

Luận điểm 3: RL là công cụ để huấn luyện và cải thiện generative models. Quá trình denoising của diffusion model có thể được mô hình hóa như một MDP nhiều bước, cho phép fine-tune model theo các mục tiêu downstream không biểu diễn được bằng likelihood (DDPO [1]). Mở rộng hơn, self-supervised RL cho phép xây dựng các Behavioral Foundation Model khám phá và nén không gian hành vi của môi trường mà không cần dữ liệu curated hay reward thủ công.

5.1.2. Tính nhất quán và mạch lạc của framework

Điểm đáng chú ý là tutorial xây dựng được một luận điểm nhất quán từ đầu đến cuối. Bắt đầu từ quan sát đơn giản rằng score function – đối tượng trung tâm của generative modeling – xuất hiện trực tiếp trong policy gradient của RL, tutorial dần mở rộng mối liên hệ này qua nhiều lớp: từ reward-weighted regression, đến dual RL, đến successor measure, rồi đến temporal contrastive learning và cuối cùng là zero-shot

RL. Mỗi bước đều có nền tảng toán học rõ ràng và kết nối trực tiếp với bước tiếp theo.

Cách trình bày này giúp người đọc không chỉ nắm bắt từng phương pháp cụ thể mà còn hiểu được *tại sao* các kết nối đó tồn tại ở mức nguyên lý, từ đó có thể tự suy ra các biến thể và mở rộng.

5.2. Ưu điểm của hướng tiếp cận

5.2.1. Loại bỏ sự phụ thuộc vào reward engineering

Một trong những nút thắt lớn nhất khi triển khai RL trong thực tế là việc thiết kế hàm reward. Reward phải vừa phản ánh đúng mục tiêu của người dùng, vừa đủ dense để tạo gradient học, vừa không bị mô hình khai thác theo cách không mong muốn. Trong nhiều ứng dụng thực tế – robot dọn phòng, phẫu thuật robot, tổng hợp hóa học – việc mã hóa “mục tiêu” thành scalar reward là gần như bất khả thi.

Khung goal-conditioned RL được trình bày trong tutorial giải quyết vấn đề này một cách triệt để. Bằng cách định nghĩa bài toán qua *dữ liệu* (quan sát mục tiêu g) thay vì scalar, người dùng chỉ cần cung cấp ví dụ về trạng thái mong muốn. Phần còn lại – bao gồm đánh giá chính sách và tối ưu – được dẫn xuất tự động từ cấu trúc xác suất. Điều này về cơ bản đưa RL đến gần hơn với phương pháp luận học máy tiêu chuẩn: bài toán được định nghĩa qua dữ liệu, không phải qua code đặc thù.

5.2.2. Mở rộng năng lực biểu diễn của policy

Việc sử dụng diffusion model hay transformer làm policy mang lại khả năng biểu diễn phân phối hành động đa mode – điều mà Gaussian policy truyền thống không làm được. Trong các bài toán robot manipulation hay tổng hợp hóa học, thường tồn tại nhiều chuỗi hành động hợp lệ để đạt cùng một mục tiêu. Policy Gaussian sẽ trung bình hóa các mode này, dẫn tới các hành động không hợp lệ. Diffusion policy tránh được vấn đề này bằng cách biểu diễn trực tiếp phân phối đa mode.

5.2.3. Khả năng học không cần giám sát và dữ liệu tự thu thập

Góc nhìn self-supervised RL (Phần IV của tutorial) giải quyết câu hỏi cơ bản: dữ liệu từ đâu mà có? Thay vì phụ thuộc vào dataset curated do con người tạo ra, agent có thể tự khám phá môi trường, thu thập dữ liệu và tổ chức kiến thức thông qua skill learning. Khung Behavioral Foundation Model [7] cho thấy rằng quá trình học như vậy có thể tạo ra các biểu diễn kỹ năng có khả năng tổng quát hóa mạnh, sẵn sàng được *prompt* để giải quyết downstream task mới – tương tự LLM nhưng trong không gian

hành vi thay vì không gian ngôn ngữ.

5.3. Hạn chế và thách thức còn tồn tại

5.3.1. Hạn chế về khả năng mở rộng (scalability)

Mặc dù có nền tảng lý thuyết vững chắc, phần lớn các phương pháp được trình bày trong tutorial vẫn được thực nghiệm trên các môi trường quy mô nhỏ hoặc trung bình (MuJoCo, Atari, maze navigation). Khi chuyển sang các bài toán thực tế với không gian trạng thái và hành động cực kỳ lớn – ví dụ robot thao tác trong môi trường chưa biết, hay điều khiển lò phản ứng hạt nhân – các thuật toán hiện tại vẫn đối mặt với thách thức về hiệu quả mẫu và độ ổn định huấn luyện.

Temporal contrastive learning và successor measure estimation đòi hỏi lượng dữ liệu lớn để ước lượng chính xác trong không gian chiều cao. Mặc dù các biểu diễn log-linear (Section 4.6 của tutorial) có một số tính chất lý thuyết tốt, tính thực tiễn ở quy mô lớn vẫn cần được kiểm chứng thêm.

5.3.2. Vấn đề phân phối lệch (distribution shift)

Phần lớn các phương pháp offline RL trong tutorial đều đối mặt với vấn đề distribution shift: policy được học từ dữ liệu của behavior policy, nhưng sau đó được triển khai theo phân phối khác. Diffusion-DICE giải quyết phần nào vấn đề này thông qua guide-then-select, nhưng vẫn phụ thuộc vào giả định rằng dataset offline đủ phong phú để phủ được vùng hành động tối ưu.

Trong setting online hay hybrid, vấn đề trở nên phức tạp hơn khi policy liên tục cập nhật và phân phối dữ liệu thay đổi theo. Các thuật toán dual RL (Section 3.4) cung cấp công cụ lý thuyết để phân tích vấn đề này, nhưng cài đặt thực tế vẫn yêu cầu nhiều kỹ thuật ổn định hóa bổ sung.

5.3.3. Phụ thuộc vào chất lượng reward model

Đối với các phương pháp kết hợp reward trong RL fine-tuning generative models (ví dụ DDPO), chất lượng toàn bộ hệ thống phụ thuộc trực tiếp vào độ chính xác của reward model. Khi reward model chỉ là một proxy hẹp cho mục tiêu thực sự của người dùng, mô hình có thể tối ưu proxy thay vì cải thiện chất lượng thực – hiện tượng *reward hacking*. Đây là vấn đề không chỉ của DDPO mà của toàn bộ paradigm RLHF (Reinforcement Learning from Human Feedback) và đang là chủ đề nghiên cứu tích cực trong cộng đồng.

5.3.4. Thách thức trong khái quát hóa (generalization)

Tutorial kết thúc bằng lời kêu gọi về vai trò của generalization trong RL (Section 6 của tutorial). Đây là một thiếu hụt hiện tại quan trọng: trong khi các mô hình ngôn ngữ và hình ảnh có thể generalize tốt sang các input chưa gặp trong training, hầu hết các RL agent vẫn bị overfitting vào môi trường và phân phối task cụ thể đã train. Skill learning và successor measure cung cấp một số công cụ để giải quyết vấn đề này, nhưng khả năng generalize của RL agent vẫn còn kém xa so với LLM hay visual generative models.

5.4. Hướng nghiên cứu tương lai

5.4.1. Tích hợp nền tảng mô hình (foundation models) với RL

Một hướng nghiên cứu hứa hẹn là tận dụng các foundation model (LLM, vision-language model) như nguồn prior kiến thức để khởi động quá trình học RL. Trong khi tutorial giới thiệu RLZero như một bước đầu tiên – dùng video generative model để tưởng tượng hành vi mục tiêu – vẫn còn nhiều câu hỏi mở về cách kết hợp language grounding, visual perception và decision-making trong một framework thống nhất.

Đặc biệt, khả năng suy luận theo ngữ cảnh dài của LLM có thể bổ sung hiệu quả cho successor measure trong việc lập kế hoạch nhiều bước. Thay vì chỉ ước lượng xác suất đến trạng thái s_f theo phân phối hình học, có thể kết hợp với khả năng lập kế hoạch ngôn ngữ để xử lý các task phức tạp hơn.

5.4.2. Cải thiện hiệu quả mẫu trong môi trường thực

Đối với các ứng dụng thực tế như robotics hay y tế, số lượng tương tác với môi trường bị giới hạn nghiêm ngặt. Các phương pháp model-based RL (Section 2.2) kết hợp với world model chất lượng cao (ví dụ diffusion-based world model) là hướng đi tự nhiên để giảm yêu cầu dữ liệu thực. Tuy nhiên, như tutorial đề cập, compounding error của world model vẫn là thách thức lớn cần các kỹ thuật phát hiện và điều chỉnh hiệu quả hơn.

5.4.3. Reward-free RL ở quy mô lớn

Skill learning và self-supervised RL (Phần IV) đặt nền móng cho RL không cần reward. Bước tiếp theo là đưa paradigm này lên quy mô lớn hơn, tương đương với scale của LLM pretraining. Câu hỏi mở bao gồm: làm thế nào thiết kế không gian skill $\mathcal{Z}$ sao

cho đủ phong phú để biểu diễn đa dạng hành vi? Làm thế nào kết hợp skill learning với instruction following để tạo ra các agent thực sự có thể nhận lệnh ngôn ngữ?

5.4.4. Lý thuyết về generalization trong RL

Như tutorial nhấn mạnh, generalization là *the next frontier* của RL. Cần xây dựng lý thuyết rõ ràng về khi nào và tại sao các biểu diễn học từ một tập task có thể transfer sang task mới. Kết quả về *horizon generalization* [7] – train trên task ngắn, giải được task dài – là một tín hiệu tích cực, nhưng vẫn còn hạn chế ở các setting đặc thù. Việc mở rộng các kết quả này sang không gian trạng thái và hành động liên tục, đa chiều là một bài toán lý thuyết quan trọng.

5.5. Nhận xét về ý nghĩa thực tiễn

Nhìn từ góc độ ứng dụng, framework được trình bày trong tutorial có ý nghĩa thực tiễn rõ ràng trong ít nhất ba lĩnh vực.

Đối với **robotics**, khả năng học goal-reaching từ video demonstration mà không cần reward (GCBC, goal-conditioned RL) và khả năng biểu diễn policy đa mode (diffusion policy) trực tiếp giải quyết hai điểm yếu lớn nhất của robot learning hiện tại: thiếu dữ liệu có nhãn reward và sự đa dạng của chuỗi hành động hợp lệ.

Đối với **LLM agents**, việc nhìn nhận chuỗi token generation như một MDP (tương tự DDPO áp dụng cho diffusion model) cung cấp nền tảng lý thuyết cho RLHF và các phương pháp fine-tuning bằng reward. Framework dual RL và successor measure còn gợi ý các cách mới để đánh giá và tối ưu khả năng lập kế hoạch của LLM mà không phụ thuộc vào human annotation.

Đối với **khoa học và kỹ thuật** (điều khiển lò phản ứng, thiết kế phân tử, tối ưu hóa quy trình công nghiệp), việc kết hợp world model chất lượng cao với thuật toán RL hiệu quả mẫu mở ra khả năng học chính sách tốt với rất ít tương tác thực với hệ thống vật lý – giảm thiểu chi phí và rủi ro trong quá trình thực nghiệm.

5.6. Kết luận

Tutorial *Generative AI Meets Reinforcement Learning* của Eysenbach và Zhang tại ICML 2025 trình bày một quan điểm thống nhất và có chiều sâu về mối quan hệ giữa hai lĩnh vực. Từ reward-weighted regression, diffusion policy, Diffusion-DICE đến temporal contrastive learning, dual RL, skill learning và zero-shot RL, mỗi chủ đề đều được gắn kết bởi một ngôn ngữ toán học chung: phân phối xác suất trên trajectory,

score function, và successor measure.

Báo cáo này đã tổng hợp và trình bày các nội dung chính của tutorial qua bốn phần: kiến thức nền về RL và generative models (Phần 2), ứng dụng generative models trong RL bao gồm world model, diffusion policy và DDPO (Phần 3), tương tác như generative model – goal-conditioned RL, dual RL, successor measure và skill learning (Phần 4), cùng với kết quả thực nghiệm minh họa (Phần 5).

Nhóm nhận thấy rằng đóng góp lớn nhất của tutorial không chỉ nằm ở các thuật toán cụ thể, mà ở chỗ nó cung cấp một *khung nhìn* giúp các nhà nghiên cứu từ cả hai phía – generative modeling và RL – có thể đọc hiểu kết quả của nhau và phát triển ý tưởng mới ở giao thoa. Khi mô hình ngôn ngữ lớn và các hệ thống agent ngày càng phát triển, sự hợp nhất giữa generative AI và reinforcement learning không còn là một hướng nghiên cứu tùy chọn mà đang trở thành nền tảng không thể thiếu.

Bảng 6: Tóm tắt các phương pháp chính và vị trí trong framework

Phương pháp	Vai trò	Đóng góp chính
Reward-weighted Regression	Dữ liệu trong RL	Kết nối RL và maximum likelihood
Diffusion Policy / Diffusion-DICE	Policy biểu diễn phong phú	Xử lý phân phối đa mode trong offline RL
DDPO	Fine-tune generative model	RL objective cho diffusion model
Goal-conditioned RL / GCBC	Loại bỏ reward engineering	Định nghĩa bài toán qua dữ liệu
Temporal Contrastive Learning	Ước lượng successor measure	Học biểu diễn không cần reward
Dual RL / DICE	Phân tích phân phối dữ liệu	Thống nhất on-policy và off-policy
Skill Learning / BFM	Khám phá và tổ quát hóa	Agent tự tạo dữ liệu và kỹ năng
RLZero	Zero-shot task inference	Điều khiển từ ngôn ngữ, không cần giám sát

Tài liệu

- [1] Kevin Black, Michael Janner, Yilun Du, Ilya Kostrikov, and Sergey Levine. Training diffusion models with reinforcement learning. In *International Conference on Learning Representations*, 2024.
- [2] Kevin Black, Michael Janner, Yilun Du, Ilya Kostrikov, and Sergey Levine. Training diffusion models with reinforcement learning: Project page, 2024.
- [3] Lili Chen, Kevin Lu, Aravind Rajeswaran, Kimin Lee, Aditya Grover, Michael Laskin, Pieter Abbeel, Aravind Srinivas, and Igor Mordatch. Decision transformer: Reinforcement learning via sequence modeling. In *Advances in Neural Information Processing Systems*, 2021.
- [4] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 2025.
- [5] Kurtland Chua, Roberto Calandra, Rowan McAllister, and Sergey Levine. Deep reinforcement learning in a handful of trials using probabilistic dynamics models. In *Advances in Neural Information Processing Systems*, 2018.
- [6] Marc Deisenroth and Carl E. Rasmussen. PILCO: A model-based and data-efficient approach to policy search. In *Proceedings of the 28th International Conference on Machine Learning*, pages 465–472, 2011.
- [7] Benjamin Eysenbach and Amy Zhang. Notes from an ICML 2025 tutorial: Generative AI meets reinforcement learning, 2025. ICML 2025 tutorial notes.
- [8] Jesse Farebrother, Matteo Pirodda, Andrea Tirinzoni, Rémi Munos, Alessandro Lazaric, and Ahmed Touati. Temporal difference flows, 2025. arXiv preprint.
- [9] Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. *arXiv preprint arXiv:1912.01603*, 2019.
- [10] Michael Janner, Yilun Du, Joshua Tenenbaum, and Sergey Levine. Planning with diffusion for flexible behavior synthesis. In *International Conference on Machine Learning*, 2022.
- [11] Michael Janner, Qiyang Li, and Sergey Levine. Offline reinforcement learning as one big sequence modeling problem. In *Advances in Neural Information Processing Systems*, 2021.

-
- [12] Liyuan Mao, Haoran Xu, Xianyuan Zhan, Weinan Zhang, and Amy Zhang. Diffusion-DICE: In-sample diffusion guidance for offline reinforcement learning. In *Advances in Neural Information Processing Systems*, 2024.
 - [13] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, 2022.
 - [14] Keiran Paster, Sheila A. McIlraith, and Jimmy Ba. You can’t count on luck: Why decision transformers and RvS fail in stochastic environments. In *Advances in Neural Information Processing Systems*, 2022.
 - [15] Jan Peters and Stefan Schaal. Reinforcement learning by reward-weighted regression for operational space control. In *Proceedings of the 24th International Conference on Machine Learning*, pages 745–750, 2007.
 - [16] Ilia Shumailov, Zakhar Shumaylov, Yiren Zhao, Nicolas Papernot, Ross Anderson, and Yarin Gal. Ai models collapse when trained on recursively generated data. *Nature*, 2024.
 - [17] Harshit Sikchi, Siddhant Agarwal, Pranaya Jajoo, Samyak Parajuli, Caleb Chuck, Max Rudolph, Peter Stone, Amy Zhang, and Scott Niekum. RL zero: Zero-shot language to behaviors without any supervision. *arXiv preprint arXiv:2412.05718*, 2024.
 - [18] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8:229–256, 1992.